From zero to the interview

Arrays and matrices in JavaScript. Each chapter uses only ideas from earlier chapters. You start by indexing a list. You finish by filling a grid in order.

Sahil · one learning path · Stay, then Then, then a second pass

00 The array
You become able to index, sort, and tell a subarray from a subsequence.

01 Read the problem
You become able to classify a prompt in four lines before any code.

02 One pass
You become able to answer each index from a running summary.

03 Two loops
You become able to enumerate every pair and every subarray.

04 Hash map
You become able to replace an inner search with a lookup.

05 Pointers from the ends
You become able to discard one end of a sorted range.

06 Same-direction pointers
You become able to overwrite an array with a read index and a write index.

07 Sliding window
You become able to grow and shrink one contiguous band.

08 Prefix sums
You become able to answer a range sum, including when negatives appear.

09 Difference array
You become able to add a value on many ranges, then read the array once.

10 A left pass and a right pass
You become able to combine two directional summaries.

11 Kadane
You become able to take the best contiguous sum, even if every value is negative.

12 Sort, then scan
You become able to pay n log n so pairs and intervals have an order.

13 Place each value
You become able to put a value at the index where it belongs.

14 Binary search
You become able to search an index, or search the answer itself.

15 Monotonic stack
You become able to find the next greater element in one pass.

16 Greedy scans
You become able to track a reach, a jump layer, or a tank.

17 Recognition
You become able to name the chapter from the wording.

18 The matrix
You become able to index rows, columns, diagonals, and four directions.

19 Walk the matrix
You become able to spiral, transpose, and rotate.

20 In-place marks
You become able to store the next state inside the cell.

21 2D prefix
You become able to sum any rectangle, then collapse a matrix to a 1D map.

22 Sorted matrix
You become able to binary-search a flat index, or walk a staircase.

23 Regions
You become able to flood a region and run a multi-source search.

24 Filling order
You become able to fill a cell only after the cells it depends on.

25 Four weeks
You become able to order the practice and sit a mixed mock.

BEFORE CHAPTER 0

How to study this book

Read a chapter, close it, and rewrite the template. The problems exist to make that template yours.

Do one chapter at a time. Read it once, including the trace. Close the page. Rewrite the template from a blank file until it matches, including the order of updates the trace depends on. Then solve the **Stay** problems in the listed order, before you open the next chapter. The **Then** problems are the same chapter, after Stay. **Second pass** problems wait until you have finished the path. First pass means Stay plus Then. Second pass means the harder tag.

When you are stuck, set a timer for 25 minutes. If you still do not have a correct approach, reread only the recognition box and the "say this" box. Do not open a solution. The recognition box is the thing you are training.

A 45-MINUTE INTERVIEW

1. Read the prompt until you can restate the input and the output.
2. Say the pattern name and the brute-force cost out loud.
3. Write the template. Narrate the invariant, not the syntax.
4. Test four inputs before you say you are done: empty, one element, duplicates, negatives. Use the ones the constraints actually allow.

Every link goes to a real LeetCode problem: the number, the official title, and leetcode.com/problems/<slug>/. Difficulty tags match the site. A Stay problem is never premium-only. A premium cousin is named in a sentence when it is the same pattern, and it is not required.

JavaScript in this book is interview JavaScript. Use **const**. Use **Map** when the key is a number or you need an index of 0, because an object key is a string and a stored index of 0 is falsy. Sort with **sort((a, b) => a - b)**. Binary search is half-open, or the comment says otherwise. Do not walk an array with **for...in**.

Carry forward. Chapter 0 is the list itself: index 0, last index n − 1, and what O(n²) costs when n is 100,000.

The array in JavaScript

An index is a position. The first is 0, the last is n − 1, and a subarray has no gaps.

An array is a row of values. Each value has a position, and that position is an integer. The first position is 0. If the array holds n values, the last position you may read is n − 1. `nums.length` is n. `nums[nums.length]` is past the end, and it is `undefined`.

You will use three words that interviews treat as different. Take **[2, 7, 3]**.

WORD	RULE	THE LIST
Subarray	Contiguous. Order stays.	[2], [2,7], [2,7,3], [7], [7,3], [3]
Subsequence	Order stays. Gaps allowed.	The six subarrays, plus [2,3]
Subset	A selection. Order does not matter.	{2,3} is one subset. Empty counts. 8 subsets.

In place means a handful of variables, not a second array of length n. Building an answer array of length n is allowed by many problems, and it is not in-place. Mutating the input and returning a length is the usual in-place contract: the first `k` slots hold the answer, and the rest may be garbage.

Cost, in one sitting

Write the cost before you write the loops. Interview arrays often have n up to 1e5, sometimes 1e6. A modern judge runs very roughly 1e8 simple operations a second. n^2 at n = 1e5 is 1e10 operations. That is about a hundred seconds. It will time out. n log n at the same n is about 1.7e6 operations. That is fine.

N UP TO	FITS	DOES NOT FIT
100	n^3, sometimes 2^n with pruning	n^4
1,000	n^2	n^3
1e5	n log n, or n	n^2
1e6	n, or n log n if the work inside is tiny	n^2

Empty and one element are part of the template, not a special puzzle. If you read `nums[0]`, decide what an empty array returns. A single element has no neighbor: a loop that looks at `i` and `i - 1` must start at 1, or it must treat index 0 as a run of length 1.

The operations you will actually call

SORT IS LEXICOGRAPHIC UNTIL YOU PASS A COMPARATOR.

```
const a = [10, 2, 1];
a.push(7);     // end, O(1) amortized. a is [10, 2, 1, 7]
a.pop();       // end, O(1). a is [10, 2, 1]
a.shift();     // front, O(n). everyone slides left
a.unshift(4);  // front, O(n). everyone slides right

const buggy = [10, 2, 1];
buggy.sort();                // lexicographic: [1, 10, 2]
const fixed = [10, 2, 1];
fixed.sort((x, y) => x - y);  // numeric: [1, 2, 10]
```

The default sort converts each value to a string. "1" comes before "10", and "10" comes before "2", because the first character decides. Any time the values are numbers, write the comparator. The same bug hides inside interval sorting later: sort by `a[0] - b[0]`, not by default.

An array is an object. Two names can refer to one array.

ASSIGNMENT COPIES THE REFERENCE.

```
const a = [1, 2];
const b = a;
b[0] = 9;          // a[0] is 9 as well
const copy = a.slice();
copy[0] = 0;       // a is unchanged
```

That fact becomes a real bug when you build a matrix. `Array(m).fill(Array(n).fill(0))` fills every row with the same inner array. Chapter 18 shows the mutation. Until then, remember: `slice` or the spread `[...a]` makes a shallow copy of one level.

A carry, because digits are just an array

PLUS ONE. [9, 9] MUST BECOME [1, 0, 0].

```
function plusOne(digits) {
  for (let i = digits.length - 1; i >= 0; i--) {
    if (digits[i] < 9) {
      digits[i] += 1;
      return digits;
    }
    digits[i] = 0;
  }
  digits.unshift(1);
  return digits;
}
```

```
[1, 2, 9]
i=2: 9, write 0, carry on
i=1: 2, write 3, return [1, 3, 0]

[9, 9]
both become 0, loop ends, unshift 1 → [1, 0, 0]
```

One pass from the right. O(n) time. The final unshift moves n digits once, which is still O(n).

A sorted array that may have been rotated has at most one place where a value is greater than the next, and the wrap from the last index back to index 0 counts as a pair. More than one drop means it is not a rotation of a non-decreasing array. You will not binary-search that shape until chapter 14. Counting the drops is enough here.

AT MOST ONE DESCENT, INCLUDING THE WRAP.

```
function check(nums) {
  let drops = 0;
  for (let i = 0; i < nums.length; i++) {
    const next = nums[(i + 1) % nums.length];
    if (nums[i] > next) drops += 1;
  }
  return drops <= 1;
}
```

TRAPS

- Reading `nums[n]` because you used `<= n` or `i <= nums.length`.
- Calling `sort()` on numbers and then trusting the order.
- Calling a new array of length n an in-place algorithm.
- Using `for...in` on an array. It walks keys, the keys are strings, and a prototype can show up.
- Treating `shift` inside a loop as free. n shifts are $O(n^2)$.

Practice

Stay problems are mechanics. Then problems are the same mechanics with a carry, a count, or a wrap.

Stay before chapter 1. Then before you leave the chapter.

1929. Concatenation of Array EASY STAY
Build an array of length 2n. ans[i] and ans[i + n] both read nums[i].

1920. Build Array from Permutation EASY STAY
ans[i] = nums[nums[i]]. Two reads. Do not confuse the index with the value.

1470. Shuffle the Array EASY STAY
The array is x1..xn y1..yn. Place them as x1,y1,x2,y2 by writing index 2*i and 2*i+1.

1512. Number of Good Pairs EASY STAY
A pair is two indices i < j with the same value. Count frequencies, then add c*(c-1)/2 for each count.

1431. Kids With the Greatest Number of Candies EASY THEN
One pass for the max, a second pass to compare candies[i] + extraCandies with that max.

1295. Find Numbers with Even Number of Digits EASY THEN
Count digits by dividing by 10, or by the length of the decimal string. Zero is one digit.

66. Plus One EASY THEN
The template above. The only new digit is a leading 1 when every digit was 9.

989. Add to Array-Form of Integer EASY THEN
Add k from the right. The carry is Math.floor(sum / 10), and k shrinks the same way.

1752. Check if Array Is Sorted and Rotated EASY THEN
Count strict drops, including the pair (last, first). The answer is drops <= 1.

REWRITE FROM MEMORY
Close the page and write the sort comparator, the plus-one loop, and the six subarrays of [2, 7, 3] from memory.

Carry forward. Chapter 1 uses this vocabulary to classify a prompt before you choose a loop.

CHAPTER 01 · BUILDS ON CHAPTER 0

How to read an array problem

Four lines before code: constraints, output, contiguous and in-place, brute cost.

You can index an array. The next skill is refusing to code until you know which later chapter the prompt belongs to. Four lines are enough. Write them on paper.

1. **Input.** What is n, roughly? Are values sorted? Can they be negative? Can they be zero?
2. **Output.** A boolean, an index, a length, a count of subarrays, or a rebuilt array?
3. **Shape.** Does the answer have to be contiguous? Does it have to be in place?
4. **Brute cost.** If you tried every pair, every subarray, or every possible answer, what is the cost, and does the constraint allow it?

Chapter 3 is the brute force those lines are pointing at. Chapters 4 through 16 exist because the fourth line usually says the brute force is too slow. You do not need those chapters yet. You need the habit of writing the four lines.

RECOGNITION

- Contiguous, and a running total is enough: you are heading toward one pass, a window, or a prefix.
- Any pair, order irrelevant, values unsorted: you are heading toward a map, or toward a sort.
- The array is already sorted, and each check lets you drop one side: you are heading toward two indices.
- The problem asks for the minimum speed, capacity, or day such that a plan works: you are heading toward searching the answer, not the index.

SAY THIS BEFORE YOU CODE

I am not choosing an algorithm yet. I am writing the input, the output, whether the answer must be contiguous, and what the brute force costs.

Eight drills

For each prompt, write the chapter you think it will land in, and one sentence why. Do not write code. The answer key is in chapter 17. Leave it closed.

1. Longest subarray whose sum is at most K. Every value is positive.
2. Count the subarrays whose sum is exactly K. Values may be negative.
3. Two indices in a sorted array add to a target.
4. Remove duplicates from a sorted array, in place.
5. For each day, how many days until a strictly warmer temperature.
6. How many islands are in a grid of land and water.
7. Add a value on many index ranges, then read the final array.
8. The minimum eating speed so the piles are finished within H hours.

TRAPS

- Starting to code because the prompt looks like one you have seen. The four lines are the check.
- Calling every contiguous problem a sliding window. Negatives, and a sum that a later negative can repair, are a prefix map. Chapter 7 and chapter 8 exist so you can tell them apart.
- Guessing the chapter from a keyword. "Maximum" is not Kadane. "Sorted" is not binary search. The four lines decide.

REWRITE FROM MEMORY

Close the page and write the four lines, then classify the eight drills again without looking.

Carry forward. Chapter 2 is the first algorithm: one pass, where the answer at i depends only on a[i] and a summary of what you have already seen.

CHAPTER 02 · BUILDS ON CHAPTER 1

One pass and running state

The answer at i is a function of nums[i] and a summary of what you have already seen.

You can read a problem. The first family of solutions walks the array once. At index i you are allowed to use `nums[i]` and a small summary of indices `0..i` or `i..n−1`. You are not allowed to rescan.

RECOGNITION

- The value you need at i is a sum, a count, a min, or a max of a prefix or a suffix.
- The decision never needs a specific earlier index, only a summary of them.
- If the summary would have to forget something in the middle and remember it later, this is not the chapter. That is a window, a prefix map, or a stack.

SAY THIS BEFORE YOU CODE

The invariant: when I finish index i, the summary describes exactly the prefix I have walked, and the answer at i is a function of that summary and nums[i].

Running sum

THE SUMMARY IS THE SUM SO FAR.

```
function runningSum(nums) {
  const out = [];
  let sum = 0;
  for (const x of nums) {
    sum += x;
    out.push(sum);
  }
  return out;
}
```

```
[1, 2, 3, 4]
sum 1 → [1]
sum 3 → [1, 3]
sum 6 → [1, 3, 6]
sum 10 → [1, 3, 6, 10]
```

O(n) time. O(n) extra for the output, which the problem requires. The summary itself is one number.

Highest altitude is the same loop. Start at altitude 0, add each gain, and keep the maximum altitude you have touched, including the start. A negative gain can pull you down. The answer is the best altitude, not the final one.

A run that resets

THE SUMMARY IS THE CURRENT RUN. A ZERO THROWS IT AWAY.

```
function findMaxConsecutiveOnes(nums) {
  let best = 0;
  let run = 0;
  for (const x of nums) {
    if (x === 1) {
      run += 1;
      if (run > best) best = run;
    } else {
      run = 0;
    }
  }
  return best;
}
```

```
[1, 1, 0, 1, 1, 1]
run 1, 2, then 0 resets, then 1, 2, 3
best = 3
```

Longest continuous increasing subsequence, consecutive characters, and large groups are this loop with a different reset rule. A group of identical characters records a start index when the run reaches length 3. A mountain is two of these loops in a row: walk up while the next value is greater, refuse a peak at either end, then walk down, and demand that you finish on the last index.

Best day to sell, one transaction

PROFIT USES YESTERDAY'S MINIMUM. THEN THE MINIMUM MAY MOVE.

```
function maxProfit(prices) {
  let minPrice = Infinity;
  let best = 0;
  for (const price of prices) {
    const profit = price - minPrice;
    if (profit > best) best = profit;
    if (price < minPrice) minPrice = price;
  }
  return best;
}
```

```
[1, 2]
correct: price 1, profit is -Infinity so best stays 0, then min = 1.
         price 2, profit = 1, best = 1.

broken, if you write the min first:
price 1, min = 1, profit = 0
price 2, min = 2, profit = 0
answer 0, and you can never sell above the min you just stored.
```

O(n) time. O(1) extra memory.

The minimum is a summary of the past. The price you are looking at is the sell. Updating the minimum first makes today both the buy and the sell.

The greatest element on the right

Sometimes the summary lives in the suffix. Walk from the last index. The answer at i is the best you have already seen to its right, and only then may that cell become the new best.

REPLACE, THEN CONSIDER THE OLD VALUE AS A CANDIDATE.

```
function replaceElements(arr) {
  let best = -1;
  for (let i = arr.length - 1; i >= 0; i--) {
    const cur = arr[i];
    arr[i] = best;
    if (cur > best) best = cur;
  }
  return arr;
}
```

```
[17, 18, 5, 4, 6, 1]
from the right, best starts at -1
1 → write -1, best = 1
6 → write 1, best = 6
4 → write 6
5 → write 6
18 → write 6, best = 18
17 → write 18
result [18, 6, 6, 6, 1, -1]
```

This is not "the next greater element." It is the greatest element anywhere to the right. The next greater element cannot be answered by one running maximum, because a smaller neighbor can still be the nearest greater one. That wait is chapter 15.

One pass after you know the total

A pivot index has an equal sum on both sides. You cannot know the right-hand sum until you know the total. Add the array once. Then walk again: the right-hand sum is `total - left - nums[i]`. Compare, and only then add `nums[i]` into `left`.

THE TOTAL TURNS ONE FUTURE SUM INTO ARITHMETIC.

```
function pivotIndex(nums) {
  let total = 0;
  for (const x of nums) total += x;
  let left = 0;
  for (let i = 0; i < nums.length; i++) {
    const right = total - left - nums[i];
    if (left === right) return i;
    left += nums[i];
  }
  return -1;
}
```

```
[1, 7, 3, 6, 5, 6], total 28
i=0 left 0, right 27, no, left becomes 1
i=1 left 1, right 20, no, left becomes 8
i=2 left 8, right 17, no, left becomes 11
i=3 left 11, right 11, return 3
```

Two passes, each O(n). O(1) extra memory. This is the bridge into prefix sums: the total is prefix[n], and the right-hand sum is prefix[n] – prefix[i+1].

TRAPS

- Updating the minimum price before the profit. [1, 2] returns 0.
- Letting a run-length answer stay 0 on a one-element array. A single 1 is a run of 1.
- Using the replaced cell as the "greatest on the right" after you have overwritten it. Save the old value first.
- Adding nums[i] into the left sum before the pivot comparison. The pivot itself belongs to neither side.

Practice

Stay before chapter 3. Then before you leave the chapter.

1480. Running Sum of 1d Array EASY STAY
The running-sum template. out[i] is the sum of the prefix ending at i.

485. Max Consecutive Ones EASY STAY
Reset the run on 0. Track the best run. Empty input returns 0.

121. Best Time to Buy and Sell Stock EASY STAY
One buy, one sell, sell after buy. Profit before you move the minimum.

1299. Replace Elements with Greatest Element on Right Side EASY STAY
Walk from the right. The last answer is -1.

724. Find Pivot Index EASY STAY
Total first, then left versus total - left - nums[i]. Return the leftmost index.

1732. Find the Highest Altitude EASY THEN
Running sum of the gains. The best altitude includes the start at 0.

674. Longest Continuous Increasing Subsequence EASY THEN
Reset the run to 1 when the value does not strictly increase. A single element is length 1.

1446. Consecutive Characters EASY THEN
The same run, compared on characters instead of bits.

830. Positions of Large Groups EASY THEN
When a run of the same character ends, record [start, end] if the length is at least 3.

896. Monotonic Array EASY THEN
Two flags: still non-decreasing, still non-increasing. Equals keep both flags alive.

941. Valid Mountain Array EASY THEN
Strict climb, then strict descent. The peak cannot be the first or the last index.

1822. Sign of the Product of an Array EASY THEN
A zero returns 0. Otherwise the sign is the parity of the negative count. Do not multiply.

1991. Find the Middle Index in Array EASY THEN
The same balance as the pivot index. Left sum equals right sum.

REWRITE FROM MEMORY

Rewrite the stock loop and the right-to-left replacement until the update order matches the traces.

Carry forward. Chapter 3 drops the summary and just writes both loops, so you can see the $O(n^2)$ algorithm that later chapters replace.

CHAPTER 03 · BUILDS ON CHAPTER 2

Two loops

When n is small, write the pairs or the subarrays directly. Reset the sum at every new start.

A summary is the wrong tool when you truly need every pair, or every contiguous slice, and n is small enough that n^2 fits. This chapter is that algorithm. It is also the algorithm chapters 7 and 8 throw away when n grows.

RECOGNITION

- n is a few hundred, or the problem is explicitly "count every pair."
- You need each pair (i, j) with i < j, or each subarray [i, j].
- There is no extra structure yet: not sorted, no complement you can store, no monotone window.

SAY THIS BEFORE YOU CODE

Brute force is the right answer when the constraint allows it. I will name the cost before I nest the loops.

Pairs, then subarrays

A pair of indices uses `j` starting at `i + 1`, so you never compare a value with itself and you never count (j, i) again. A subarray is different. You pick a start `i`, set the sum back to 0, and extend the end `j` from `i` forward. Forgetting to reset the sum makes the second start inherit the first start's total.

RESET THE SUM AT EVERY START. THIS IS THE ALGORITHM CHAPTER 8 REPLACES.

```
function countSubarraysWithSum(nums, k) {
  let count = 0;
  for (let i = 0; i < nums.length; i++) {
    let sum = 0;
    for (let j = i; j < nums.length; j++) {
      sum += nums[j];
      if (sum === k) count += 1;
    }
  }
  return count;
}
```

```
nums = [1, 1, 1], k = 2
i=0 sum 1, 2 (count), 3
i=1 sum 1, 2 (count)
i=2 sum 1
count = 2, the subarrays [1, 1] starting at 0 and at 1

number of subarrays in an array of length n: n * (n + 1) / 2
n = 3 → 6 subarrays, and you just visited all 6
```

Pairs are $O(n^2)$. Subarrays are $O(n^2)$. Three nested index loops are $O(n^3)$. At n = 100, n^3 is a million iterations and usually fine. At n = 1e5 it is not.

Final prices with a discount, solved now, is the pair loop: for each i, scan j > i for the first price less than or equal to `prices[i]`. The same question becomes a stack in chapter 15, because "the next smaller price" is exactly the next-smaller problem. Write the pair version until the jump is obvious.

TRAPS

- Starting the inner pair loop at j = 0 and then skipping i === j. You will double-count or you will use a value twice.
- Not resetting the subarray sum when i moves.
- Reaching for a hash map on a problem whose n is 50. The nested loops are shorter and they are correct.

Practice

Stay before chapter 4. Then before you leave the chapter.

1365. How Many Numbers Are Smaller Than the Current Number EASY STAY

For each i, count j where nums[j] < nums[i]. n is at most 500. The pair loops are the point.

1588. Sum of All Odd Length Subarrays EASY STAY

Enumerate subarrays, add the sum when the length is odd. Later you can also count how many odd-length subarrays cover each index.

1534. Count Good Triplets EASY THEN

Three loops, i < j < k, and three absolute-value checks. n is at most 100.

1475. Final Prices With a Special Discount in a Shop EASY THEN

For each i, the first j > i with prices[j] <= prices[i]. Brute force now. The stack version is chapter 15.

REWRITE FROM MEMORY

Rewrite the double loop that counts subarrays with sum k, including the reset, from a blank file.

Carry forward. Chapter 4 takes the inner loop that asks "have I seen the complement?" and answers it with a map.

CHAPTER 04 · BUILDS ON CHAPTERS 2 AND 3

Hash map

The inner loop asked whether you have seen the complement. Look that up, then insert.

The inner loop in chapter 3 often asks one question: have I already seen the value that would finish this pair? A map answers that in expected constant time. The key is the value. The stored thing is an index, a count, or a list.

RECOGNITION

- Unsorted, and you need the complement target − x, or you need to know whether x has occurred.
- You need a frequency, a last index, or the group of items that share a key.
- You need the longest run of consecutive values, and the array order is irrelevant.

SAY THIS BEFORE YOU CODE

I will look up the complement first. I will insert the current value after that lookup, so a value is never paired with itself.

Two sum

MAP, NOT AN OBJECT. LOOKUP, THEN INSERT.

```
function twoSum(nums, target) {
  const seen = new Map();
  for (let i = 0; i < nums.length; i++) {
    const need = target - nums[i];
    if (seen.has(need)) return [seen.get(need), i];
    seen.set(nums[i], i);
  }
  return [];
}
```

```
nums = [3], target = 6
need = 3, map is empty, no hit
then insert 3 → index 0
a single 3 is not used twice

nums = [2, 7, 11, 15], target = 9
i=0 insert 2
i=1 need 2, hit index 0, return [0, 1]

the object bug: if (obj[need]) is false when the stored index is 0
Map.has does not care that the index is 0
```

Expected O(n) time. O(n) extra memory.

Use a **Map** for numeric keys. An object turns keys into strings, and a truthiness check treats index 0 as missing. `freq.set(x, (freq.get(x) ?? 0) + 1)` is the frequency update. `??` only replaces `undefined`. A **Set** is the same idea when you store membership and nothing else.

What the map stores

- **Index of the complement.** Two sum. Insert after the lookup.
- **Last index.** Contains duplicate II: if this value was seen at j and `i - j <= k`, return true, then store i.
- **Frequency.** Anagrams, good pairs, intersections that must respect duplicates. Intersection II walks one array's counts down as you accept values from the other.
- **A group.** Group anagrams: the key is the word's characters sorted and joined. Every word with that key shares a list.

THE SORTED SPELLING IS THE KEY. THE ORIGINAL SPELLING IS THE VALUE.

```
function groupAnagrams(strs) {
  const groups = new Map();
  for (const s of strs) {
    const key = s.split("").sort().join("");
    if (!groups.has(key)) groups.set(key, []);
    groups.get(key).push(s);
  }
  return [...groups.values()];
}
```

Longest consecutive run

Put every value in a set. A run should be counted once, from its start. The start is a value `x` whose predecessor `x − 1` is absent. From there, walk `x, x+1, x+2` while the set contains them. Across the whole algorithm each number is visited by the inner walk at most once, so the total time stays linear even though a while loop sits inside a for loop.

ONLY A NUMBER WITH NO PREDECESSOR STARTS A RUN.

```
function longestConsecutive(nums) {
  const set = new Set(nums);
  let best = 0;
  for (const x of set) {
    if (set.has(x - 1)) continue;
    let y = x;
    while (set.has(y)) y += 1;
    if (y - x > best) best = y - x;
  }
  return best;
}
```

```
[100, 4, 200, 1, 3, 2]
100 starts a run of length 1 (99 is absent)
4 does not start (3 is present)
200 starts a run of length 1
1 starts: 1, 2, 3, 4 → length 4
3 and 2 do not start
best = 4
```

Expected O(n) time. O(n) extra memory.

The O(1) memory upgrade, after you can count

Majority element, problem 169, promises that one value occurs more than n/2 times. You can count with a map. Boyer–Moore does it with two variables. Keep a candidate and a vote. A match increments. Anything else decrements. When the vote hits 0, the next value becomes the candidate. Pairs of "candidate, other" cancel. The promised majority cannot be cancelled away.

169 PROMISES A MAJORITY, SO THE CANDIDATE IS THE ANSWER. NO SECOND PASS.

```
function majorityElement(nums) {
  let candidate = 0;
  let votes = 0;
  for (const x of nums) {
    if (votes === 0) candidate = x;
    votes += x === candidate ? 1 : -1;
  }
  return candidate;
}
```

```
[2, 2, 1, 1, 1, 2, 2]
2 votes 1
2 votes 2
1 votes 1
1 votes 0
1 becomes candidate, votes 1
2 votes 0
2 becomes candidate, votes 1
answer 2
```

Problem 229 asks for every value that occurs more than n/3 times. There are at most two. Keep two candidates. The problem does *not* promise that a surviving candidate is actually frequent enough, so a second pass counts the two survivors and keeps only those whose real count exceeds n/3.

TWO CANDIDATES. THE SECOND PASS IS REQUIRED.

```
function majorityElement(nums) {
  let a = 0;
  let b = 0;
  let ca = 0;
  let cb = 0;
  for (const x of nums) {
    if (x === a) ca += 1;
    else if (x === b) cb += 1;
    else if (ca === 0) { a = x; ca = 1; }
    else if (cb === 0) { b = x; cb = 1; }
    else { ca -= 1; cb -= 1; }
  }
  ca = 0;
  cb = 0;
  for (const x of nums) {
    if (x === a) ca += 1;
    else if (x === b) cb += 1;
  }
  const out = [];
  if (ca > nums.length / 3) out.push(a);
  if (cb > nums.length / 3) out.push(b);
  return out;
}
```

Top K frequent values can go in buckets: index is the frequency, the bucket holds the values. Walk the buckets from the high end until you have k values. A heap also works. The bucket version is linear after the frequency map. 4Sum II splits four arrays into two pairs. Map every sum of a pair from the first two arrays, then ask the map for the negation of each pair from the last two.

TRAPS

- Inserting before the lookup. [3] with target 6 returns a pair that uses 3 twice.
- Writing `if (seen[need])` on an object. Index 0 is there and the check still fails.
- Starting a consecutive run at every number. The set still finds the answer, and the time becomes quadratic on a long run.
- Skipping the second pass on majority II. A cancelled candidate can survive with a count that is not above n/3.
- Using a running vote as the answer to 169 when you also need the count. 169 does not ask for the count. If a follow-up removes the promise, count the candidate.

Practice

Stay before chapter 5. Then before you leave the chapter. Majority II is a second pass.

1. Two Sum EASY STAY
Lookup the complement, then insert. Exactly one answer exists. You may not use the same element twice.

217. Contains Duplicate EASY STAY
A set of values seen so far. A repeat returns true.

242. Valid Anagram EASY STAY
One frequency map built from s and consumed by t, or two maps compared. Lengths must match.

128. Longest Consecutive Sequence MEDIUM STAY
Start a run only when x − 1 is absent. Duplicates disappear inside the set.

169. Majority Element EASY STAY
The vote. The promise means you return the candidate.

219. Contains Duplicate II EASY THEN
Store the last index. The test is i − j <= k.

349. Intersection of Two Arrays EASY THEN
A set of the first array. Emit a value from the second only once.

350. Intersection of Two Arrays II EASY THEN
Frequencies. Each accepted copy consumes one count.

1207. Unique Number of Occurrences EASY THEN
Map value to count, then a set of those counts. The set must be the same size as the map.

2215. Find the Difference of Two Arrays EASY THEN
Two sets. A value belongs to the first answer list when it is missing from the second set.

49. Group Anagrams MEDIUM THEN
Key by the sorted characters. The original strings stay intact inside the lists.

347. Top K Frequent Elements MEDIUM THEN
Frequency map, then buckets by count, then walk from the highest count.

454. 4Sum II MEDIUM THEN
Map sums of A[i] + B[j]. For each C[k] + D[l], add the count of the negation.

229. Majority Element II MEDIUM SECOND PASS
At most two candidates, then a real count. Keep a value only when count > n/3.

REWRITE FROM MEMORY
Rewrite two sum and the consecutive-run start rule until you insert after the lookup without thinking.

Carry forward. Chapter 5 uses two indices on a sorted array, so the complement search no longer needs a map.

CHAPTER 05 · BUILDS ON CHAPTERS 3 AND 4

Two pointers from the ends

On a sorted array, or a palindrome, each step can discard one end. A pointer here is an index.

A pointer, the first time the word shows up in this book, is an index stored in an integer. It is not a C pointer, and it does not hold an address. Two pointers from the ends means two indices, `l` on the left and `r` on the right.

You can already find a complement with a map. If the array is sorted, the complement has a direction. A sum that is too small can only be repaired by moving `l` right. A sum that is too large can only be repaired by moving `r` left. Every pair you have not checked still sits between `l` and `r`.

RECOGNITION

- The array is sorted, or you are allowed to sort it and you do not need original indices.
- You compare the two ends: a sum, a container, the larger square, a palindrome.
- Each step discards an end forever. If both ends must move only rightward, that is chapter 6 or chapter 7.

SAY THIS BEFORE YOU CODE

The invariant: every unchecked pair is still between l and r. The move I am about to make discards only pairs that cannot beat the best I already know.

Two sum on a sorted array

TOO SMALL MOVES L. TOO LARGE MOVES R. THIS PROBLEM'S ANSWER IS 1-BASED.

```
function twoSum(numbers, target) {
  let l = 0;
  let r = numbers.length - 1;
  while (l < r) {
    const sum = numbers[l] + numbers[r];
    if (sum === target) return [l + 1, r + 1];
    if (sum < target) l += 1;
    else r -= 1;
  }
  return [];
}
```

```
[2, 7, 11, 15], target 9
l=0 r=3 sum 17 → r = 2
l=0 r=2 sum 13 → r = 1
l=0 r=1 sum 9 → return [1, 2]
```

O(n) time after the array is sorted. O(1) extra memory.

The container

Width only shrinks as the ends move in. The height of a container is the shorter wall. Suppose the left wall is shorter. Every other container that still uses this left wall pairs it with some index strictly inside, so the width is smaller, and the height is still at most this left wall. None of those inner pairs can beat the area you just measured. Move the shorter side. When the walls are equal, moving either side is safe for the same reason: that height cannot improve with a smaller width.

MOVE THE SHORTER WALL ONLY.

```
function maxArea(height) {
  let l = 0;
  let r = height.length - 1;
  let best = 0;
  while (l < r) {
    const h = Math.min(height[l], height[r]);
```

```
    const area = h * (r - l);
    if (area > best) best = area;
    if (height[l] < height[r]) l += 1;
    else r -= 1;
  }
  return best;
}
```

```
[1, 8, 6, 2, 5, 4, 8, 3, 7]
l=0 (1), r=8 (7), area 8, left is shorter, l = 1
l=1 (8), r=8 (7), area 49, right is shorter, r = 7
keep going. 49 is the best.
moving the taller wall first would skip this pair and never recover it.
```

O(n) time. O(1) extra memory.

Squares, written from the back

A sorted array of negatives and positives has its largest squares at the ends. Read whichever end has the larger absolute value, and write it at the back of the output. The output index walks from n − 1 down to 0.

THE NEXT OUTPUT CELL IS THE LARGER OF THE TWO END SQUARES.

```
function sortedSquares(nums) {
  const n = nums.length;
  const out = new Array(n);
  let l = 0;
  let r = n - 1;
  for (let i = n - 1; i >= 0; i--) {
    if (Math.abs(nums[l]) > Math.abs(nums[r])) {
      out[i] = nums[l] * nums[l];
      l += 1;
    } else {
      out[i] = nums[r] * nums[r];
      r -= 1;
    }
  }
  return out;
}
```

```
[-4, -1, 0, 3, 10]
write 100 from the right, then 16 from the left, then 9, then 1, then 0
[0, 1, 9, 16, 100]
```

A palindrome is the same two indices

SKIP NON-LETTERS WITH A NAMED HELPER.

```
function isAlnum(ch) {
  return (ch >= "a" && ch <= "z") || (ch >= "0" && ch <= "9");
}

function isPalindrome(s) {
  let l = 0;
  let r = s.length - 1;
  while (l < r) {
    const L = s[l].toLowerCase();
    const R = s[r].toLowerCase();
    if (!isAlnum(L)) { l += 1; continue; }
    if (!isAlnum(R)) { r -= 1; continue; }
    if (L !== R) return false;
    l += 1;
    r -= 1;
  }
  return true;
}
```

Backspace compare can use the same shape from the ends. Walk each string backward. A # increases a skip count. A real character spends one skip, or it is the next character to compare. You never build the edited string.

EACH CALL RETURNS THE NEXT REAL CHARACTER AT OR BEFORE I.

```
function nextIndex(s, i) {
  let skip = 0;
  while (i >= 0) {
    if (s[i] === "#") {
      skip += 1;
      i -= 1;
    } else if (skip > 0) {
      skip -= 1;
      i -= 1;
    } else {
      return i;
    }
  }
  return -1;
}

function backspaceCompare(s, t) {
  let i = s.length - 1;
  let j = t.length - 1;
  while (i >= 0 || j >= 0) {
    i = nextIndex(s, i);
    j = nextIndex(t, j);
    if (i < 0 && j < 0) return true;
    if (i < 0 || j < 0 || s[i] !== t[j]) return false;
    i -= 1;
    j -= 1;
  }
  return true;
}
```

TRAPS

- Moving both ends because the area felt small. Only the shorter wall is provably useless.
- Returning a 0-based pair for two sum II. That problem wants 1-based indices.
- Writing squares from the front. The largest square is at an end, so the first cell of the answer is not known yet.
- Collapsing the palindrome filter into a chain of replaces you cannot read out loud. The helper is the interview version.
- Letting l and r cross. The loop condition is l < r, because one index is not a pair.

Practice

Stay before chapter 6. Then before you leave the chapter.

167. Two Sum II - Input Array Is Sorted MEDIUM STAY

Sorted, exactly one solution, answer is 1-based. Too small moves l.

11. Container With Most Water MEDIUM STAY

Move the shorter side only. Width is r − l, height is the min of the two walls.

977. Squares of a Sorted Array EASY STAY

Write from index n − 1. Compare absolute values at the ends.

125. Valid Palindrome EASY STAY

Skip non-alphanumeric characters. Compare lowercase letters and digits.

344. Reverse String EASY THEN

Swap s[l] and s[r], then move both ends in. The input is a character array.

345. Reverse Vowels of a String EASY THEN

Same swap, but an end moves alone until it sits on a vowel.

680. Valid Palindrome II EASY THEN

On the first mismatch, test the slice that skips l, and the slice that skips r.

844. Backspace String Compare EASY THEN

The skip count from the right. A stack of kept characters is the version you can write if you blank on the skip.

REWRITE FROM MEMORY

Rewrite two sum II and the container loop. Say why the shorter wall cannot win with a line inside it.

Carry forward. Chapter 6 keeps two indices, and both of them move to the right. One reads, one writes.

CHAPTER 06 · BUILDS ON CHAPTER 5

Same-direction two pointers

r reads every cell. w writes, and w moves only when the cell is kept.

You can move the ends of a sorted range toward each other. A different pair of indices moves in the same direction. `r` reads every cell. `w` writes, and `w` advances only when the cell you just read is a cell you are keeping. The kept prefix is `nums[0..w)`.

RECOGNITION

- Sorted, and you overwrite duplicates in place.
- You filter values out, or you slide zeros to the end, and the relative order of the kept values matters.
- You merge two sorted arrays into one buffer that already has room at the end.
- You check whether one sequence appears inside another in order. That is one index per sequence, both moving forward.

SAY THIS BEFORE YOU CODE

The invariant: r has consumed the prefix up to r, and nums[0..w) is exactly the kept version of that prefix.

Read r, write w

W ADVANCES ONLY WHEN THE VALUE STAYS.

```
function removeElement(nums, val) {
  let w = 0;
  for (let r = 0; r < nums.length; r++) {
    if (nums[r] !== val) {
      nums[w] = nums[r];
      w += 1;
    }
  }
  return w;
}
```

```
[3, 2, 2, 3], remove 3
r=0 skip
r=1 write 2 at 0, w = 1
r=2 write 2 at 1, w = 2
r=3 skip
return 2, prefix [2, 2, ...]
```

Remove duplicates from a sorted array is the same loop, except the value stays when it differs from `nums[w − 1]`. The first cell is always kept, so `w` starts at 1 when the array is not empty. Allowing two copies compares against `nums[w − 2]`: there are not yet two kept copies, or the new value is not a third copy of the value sitting two slots back.

ONE COPY COMPARES WITH W − 1. TWO COPIES COMPARE WITH W − 2.

```
function removeDuplicates(nums) {
  if (nums.length === 0) return 0;
  let w = 1;
  for (let r = 1; r < nums.length; r++) {
    if (nums[r] !== nums[w - 1]) {
      nums[w] = nums[r];
      w += 1;
    }
  }
  return w;
}
```

```
function removeDuplicatesII(nums) {
  let w = 0;
  for (let r = 0; r < nums.length; r++) {
    if (w < 2 || nums[r] !== nums[w - 2]) {
      nums[w] = nums[r];
      w += 1;
    }
  }
  return w;
}
```

```
[1, 1, 1, 2, 2, 3], at most two copies
r=0 write, w=1
r=1 write, w=2
r=2 value 1 equals nums[0], skip
r=3 value 2, write, w=3
r=4 value 2, nums[w-2] is 1, write, w=4
r=5 value 3, write, w=5
prefix [1, 1, 2, 2, 3]
```

Zeros, parity, merge from the end

Move zeroes writes the non-zeros with the same `w`, then fills the tail with zeros. Sort by parity swaps each even number into the next write slot so the evens occupy a prefix. Merging two sorted arrays into `nums1`, which already has n empty slots at the end, writes from index `m + n − 1`. The larger tail value goes in. Writing from the front would overwrite a `nums1` value you have not merged yet.

THE EMPTY TAIL IS THE SPACE YOU WRITE INTO.

```
function merge(nums1, m, nums2, n) {
  let i = m - 1;
  let j = n - 1;
  let w = m + n - 1;
  while (j >= 0) {
    if (i >= 0 && nums1[i] > nums2[j]) {
      nums1[w] = nums1[i];
      i -= 1;
    } else {
      nums1[w] = nums2[j];
      j -= 1;
    }
    w -= 1;
  }
}
```

```
nums1 = [1, 2, 3, 0, 0, 0], m = 3
nums2 = [2, 5, 6], n = 3
write 6, then 5, then 3, then 2 (from nums2), then 2 (from nums1), then 1
[1, 2, 2, 3, 5, 6]
when j is exhausted, the leftover nums1 prefix is already in place
```

Is subsequence is the one-direction pair on two strings. `i` walks the pattern, `j` walks the text. On a match, advance `i`. Always advance `j`. The pattern is a subsequence when `i` reaches its length.

Every template here is O(n) time, or O(m + n) for the merge, and O(1) extra memory besides the output the problem already gave you.

TRAPS

- Advancing w on a value you meant to delete.
- Comparing duplicates against `nums[r - 1]` only. That works for one copy. For two copies you must look at the kept prefix, at w − 2, because r's neighbor may be a third duplicate you already skipped.
- Merging 88 from index 0. The empty slots are at the end, so the safe write cursor starts at the end.
- Filling moved zeros by swapping blindly and breaking the order of non-zeros. Write non-zeros forward, then set the tail.

Practice

Stay before chapter 7. Then before you leave the chapter.

27. Remove Element EASY STAY
Write every value that is not val. Return the new length.

26. Remove Duplicates from Sorted Array EASY STAY
Sorted. Keep a value when it differs from the last kept value.

283. Move Zeroes EASY STAY
Pack the non-zeros, then fill zeros. Relative order stays.

88. Merge Sorted Array EASY STAY
Write from m + n − 1. Stop when nums2 is exhausted.

80. Remove Duplicates from Sorted Array II MEDIUM THEN
Keep the value when w < 2 or it differs from nums[w − 2].

905. Sort Array By Parity EASY THEN
Swap evens into the next write slot. Odds fall behind.

922. Sort Array By Parity II EASY THEN
Even indices want evens. Walk an even cursor and an odd cursor, and swap when both are wrong.

392. Is Subsequence EASY THEN
Advance the pattern index only on a match. The text index always advances.

REWRITE FROM MEMORY

Rewrite remove-duplicates and the merge-from-the-end loop from a blank file.

Carry forward. Chapter 7 uses the same two indices, and now both of them move right in order to bound a window.

CHAPTER 07 · BUILDS ON CHAPTER 6

Sliding window

Both indices move right. Expand with r, and shrink with while, when growing only makes the window worse.

You have two indices that can both move right. A window is the band `nums[l..r]`, inclusive. `r` grows the band. `l` shrinks it. Use a window when making the band larger only moves you toward "invalid," and shrinking from the left repairs it. Sums of non-negative numbers behave that way: adding a value cannot make the sum smaller, and dropping the left value cannot make it larger. If a negative value can repair a sum you already passed, this is chapter 8.

RECOGNITION

- Contiguous. The constraint is a sum, a count of zeros, a product, or a number of distinct values.
- Expanding moves toward invalid. Shrinking moves back toward valid. That is the monotone property.
- Values that the condition cares about are non-negative, or they are counts you can undo when they leave the left edge.
- If you need the maximum inside the window after the maximum itself slides out, a single variable cannot do it. That is a deque in chapter 15.

SAY THIS BEFORE YOU CODE

The invariant: r is the end I just included. l is the smallest start that keeps [l, r] valid. The length is r − l + 1.

Fixed length

A window of length k never hunts for l. When r advances, subtract the element that fell out, at index `r - k`, and add `nums[r]`.

SUBTRACT THE CELL THAT JUST LEFT THE WINDOW.

```
function findMaxAverage(nums, k) {
  let sum = 0;
  for (let i = 0; i < k; i++) sum += nums[i];
  let best = sum;
  for (let r = k; r < nums.length; r++) {
    sum += nums[r];
    sum -= nums[r - k];
    if (sum > best) best = sum;
  }
  return best / k;
}
```

```
[1, 12, -5, -6, 50, 3], k = 4
first window sum = 1 + 12 + (-5) + (-6) = 2
next: drop 1, add 50, sum = 51
next: drop 12, add 3, sum = 42
best sum 51, average 12.75
```

O(n) time. O(1) extra memory.

Variable length, shrunk with while

Grow with `r`. While the window is illegal, move `l`. The word is while, not if. One step of `l` may leave the window illegal.

NON-NEGATIVE VALUES. RECORD THE LENGTH BEFORE YOU SHRINK.

```
function minSubArrayLen(target, nums) {
  let l = 0;
```

```
  let sum = 0;
  let best = Infinity;
  for (let r = 0; r < nums.length; r++) {
    sum += nums[r];
    while (sum >= target) {
      const len = r - l + 1;
      if (len < best) best = len;
      sum -= nums[l];
      l += 1;
    }
  }
  return best === Infinity ? 0 : best;
}
```

```
[2, 3, 1, 2, 4, 3], target 7
r=3, window [2, 3, 1, 2], sum 8, length 4
  drop 2, sum 6, stop. best = 4
r=4, add 4, window [3, 1, 2, 4], sum 10, length 4
  drop 3, sum 7, length 3
  drop 1, sum 6, stop. best = 3
r=5, add 3, window [2, 4, 3], sum 9, length 3
  drop 2, sum 7, length 2
  drop 4, sum 3, stop. best = 2
answer 2
```

Max consecutive ones III counts zeros inside the window and shrinks while the zero count exceeds k. The answer is the window length, because a flip keeps the cell. Longest subarray of 1s after deleting one element is the same window with at most one zero, and the answer is the window length minus 1. The deletion is mandatory, even when the window contains no zero: an all-ones array of length n returns n − 1.

R − L IS (R − L + 1) − 1. ONE DELETION IS REQUIRED.

```
function longestSubarray(nums) {
  let l = 0;
  let zeros = 0;
  let best = 0;
  for (let r = 0; r < nums.length; r++) {
    if (nums[r] === 0) zeros += 1;
    while (zeros > 1) {
      if (nums[l] === 0) zeros -= 1;
      l += 1;
    }
    const len = r - l;
    if (len > best) best = len;
  }
  return best;
}
```

A repeat must not pull l backward

JUMP L ONLY WHEN THE PREVIOUS INDEX IS STILL INSIDE THE WINDOW.

```
function lengthOfLongestSubstring(s) {
  const last = new Map();
  let l = 0;
  let best = 0;
  for (let r = 0; r < s.length; r++) {
    const ch = s[r];
    if (last.has(ch) && last.get(ch) >= l) {
      l = last.get(ch) + 1;
    }
    last.set(ch, r);
    const len = r - l + 1;
    if (len > best) best = len;
  }
```

```
  return best;
}
```

```
"abba"
r=0 a, l=0, len 1
r=1 b, l=0, len 2
r=2 b, previous b is 1, which is >= l, so l = 2, len 1
r=3 a, previous a is 0, which is < l, leave l alone, len 2
best 2
if you moved l back to 1 you would claim "bba", which repeats b
```

Fruit into baskets is the same shape with a frequency map: shrink while the map holds more than two keys, and delete a key whose count hits 0.

Counting subarrays inside a valid window

Once `[l, r]` is valid, and every subarray of a valid window is valid, each start from `l` through `r` makes a valid subarray that ends at `r`. There are `r - l + 1` of them. Add that number. Do not enumerate them.

POSITIVE INTEGERS. DIVIDING BY NUMS[L] UNDOES THE MULTIPLY.

```
function numSubarrayProductLessThanK(nums, k) {
  if (k <= 1) return 0;
  let l = 0;
  let prod = 1;
  let count = 0;
  for (let r = 0; r < nums.length; r++) {
    prod *= nums[r];
    while (prod >= k) {
      prod /= nums[l];
      l += 1;
    }
    count += r - l + 1;
  }
  return count;
}
```

Exactly K is two "at most" counts. `atMost(K)` counts subarrays with sum at most K, or with at most K distinct values, or with at most K odd numbers. Exactly K is `atMost(K) - atMost(K - 1)`. Binary subarrays with sum, and nice subarrays, are this subtraction. Treat an odd number as a 1 if you are counting odds.

EXACTLY K, FOR 0/1 ARRAYS. THE VALUES ARE NON-NEGATIVE, SO THE WINDOW IS LEGAL.

```
function numSubarraysWithSum(nums, goal) {
  const atMost = (limit) => {
    if (limit < 0) return 0;
    let l = 0;
    let sum = 0;
    let count = 0;
    for (let r = 0; r < nums.length; r++) {
      sum += nums[r];
      while (sum > limit) {
        sum -= nums[l];
        l += 1;
      }
      count += r - l + 1;
    }
    return count;
  };
  return atMost(goal) - atMost(goal - 1);
}
```

Taking cards from the ends

You take k cards from the two ends. That is the same as leaving a middle band of length `n − k` untouched. The best score is the total minus the minimum sum of a middle window of that length. If k equals n, the middle is empty and the answer is the total.

Minimum operations to reduce x to zero, when every value is positive, is the same idea from the other side. You remove a prefix plus a suffix whose sum is x. The leftover middle is the longest subarray whose sum is `total − x`. The answer is `n` minus that length, or −1 if the target never appears. A negative `total − x` is impossible.

A fixed window that is a multiset

Permutation in string and find-all-anagrams share one window of length `p.length`. Count what `p` needs. Slide over `s`. A character becomes "matched" when its window count equals its needed count, and it becomes unmatched when you are about to drop it and the counts are still equal. When the number of matched characters equals the number of distinct characters in `p`, the window is an anagram. Check the equality before you decrement the leaving character.

What this chapter refuses to pretend

The maximum of a sliding window cannot live in one variable. When that maximum slides out of the left edge, the variable does not know the second maximum. Chapter 15 keeps a deque of candidate indices. Minimum window substring is the variable window on character counts, and it is a second-pass problem: shrink while the window still covers every required character, and remember the shortest span. Subarrays with K different integers is `atMost(K) − atMost(K − 1)` on a frequency map. Both templates are below so the second pass has a correct function to adapt.

SECOND PASS. MISSING COUNTS REQUIRED COPIES, NOT DISTINCT LETTERS.

```
function minWindow(s, t) {
  const need = new Map();
  for (const ch of t) need.set(ch, (need.get(ch) ?? 0) + 1);
  let missing = t.length;
  let bestL = 0;
  let bestLen = Infinity;
  let l = 0;
  for (let r = 0; r < s.length; r++) {
    if (need.has(s[r])) {
      need.set(s[r], need.get(s[r]) - 1);
      if (need.get(s[r]) >= 0) missing -= 1;
    }
    while (missing === 0) {
      const len = r - l + 1;
      if (len < bestLen) {
        bestLen = len;
        bestL = l;
      }
      if (need.has(s[l])) {
        need.set(s[l], need.get(s[l]) + 1);
        if (need.get(s[l]) > 0) missing += 1;
      }
      l += 1;
    }
  }
  return bestLen === Infinity ? "" : s.slice(bestL, bestL + bestLen);
}
```

A window pass is O(n). A frequency window is O(n) expected, with a map whose size is the alphabet you actually touch.

TRAPS

- Shrinking with if. One move of l may not restore the invariant.
- Using a window on a sum that negatives can repair. [2, -1, 2] looking for sum 3 is a prefix problem if you need every subarray, because the -1 makes a later sum smaller.
- Length written as r − l. That is the deletion-adjusted length. The ordinary length is r − l + 1.
- Moving l backward to a previous duplicate that is already outside the window.
- Believing one variable tracks the window maximum after the max leaves. It does not.

Practice

Stay before chapter 8. Then before you leave the chapter. The two hard problems are a second pass.

---

643. Maximum Average Subarray I EASY STAY

Fixed k. Add the new cell, subtract the cell that fell out.

---

209. Minimum Size Subarray Sum MEDIUM STAY

Positive numbers. Shrink while the sum is at least the target. Return 0 if none.

1004. Max Consecutive Ones III MEDIUM STAY

At most k zeros in the window. The answer is the window length, flips included.

---

3. Longest Substring Without Repeating Characters MEDIUM STAY

Jump l to lastIndex + 1 only when lastIndex is still >= l.

---

713. Subarray Product Less Than K MEDIUM STAY

Positive integers. Each valid window ending at r contributes r − l + 1.

1456. Maximum Number of Vowels in a Substring of Given Length MEDIUM THEN

Fixed k, but the sum is a vowel count: add 1 or 0, subtract the leaving 1 or 0.

1493. Longest Subarray of 1's After Deleting One Element MEDIUM THEN

At most one zero. Answer is window length minus 1.

---

904. Fruit Into Baskets MEDIUM THEN

At most two distinct values. Delete a key when its count hits 0.

---

930. Binary Subarrays With Sum MEDIUM THEN

Exactly goal, on 0/1 values: atMost(goal) − atMost(goal − 1).

1248. Count Number of Nice Subarrays MEDIUM THEN

Exactly k odd numbers. Treat odd as 1 and reuse the subtraction.

---

567. Permutation in String MEDIUM THEN

Fixed window of length s1. Compare counts, or track how many characters are matched.

---

438. Find All Anagrams in a String MEDIUM THEN

The same window. Record the start index every time the window matches.

1423. Maximum Points You Can Obtain from Cards MEDIUM THEN

Total minus the minimum middle window of length n − k.

1658. Minimum Operations to Reduce X to Zero MEDIUM THEN

Positive nums. Longest subarray with sum total − x, then n minus that length.

76. Minimum Window Substring HARD SECOND PASS

Shrink while every required character is covered. Keep the shortest span.

992. Subarrays with K Different Integers HARD SECOND PASS

atMost(K) − atMost(K − 1) with a frequency map of values.

REWRITE FROM MEMORY

Rewrite the minimum-length window and the exactly-K subtraction. Say the length formula out loud: r − l + 1.

Carry forward. Chapter 8 answers the sum questions a window must refuse: negatives, and counting every subarray whose sum is K.

CHAPTER 08 · BUILDS ON CHAPTERS 2 AND 7

Prefix sums

prefix[i] is the sum of the first i elements. Ask for sum − k before you record sum.

A window failed you when a negative number could repair a sum, or when you needed every subarray equal to K rather than the longest one. The prefix array answers both. Let `prefix[0] = 0`. Let `prefix[i]` be the sum of the first i elements, which is `nums[0] + … + nums[i − 1]`. The sum of the inclusive range `i..j` is `prefix[j + 1] − prefix[i]`.

RECOGNITION

- Sum of a range, asked many times. Build the prefix once, answer each question in O(1).
- Count subarrays whose sum is K, and values may be negative or zero.
- Longest subarray with sum K, again with negatives: store the first index of each prefix, not the last.
- A remainder. The key is the prefix modulo k, repaired so it is never negative.

SAY THIS BEFORE YOU CODE

The invariant: the map holds prefixes I have already finished. I ask for sum − k before I record the prefix I am standing on.

The array, then the map

SEED 0 → 1. QUERY, THEN INCREMENT.

```
function buildPrefix(nums) {
  const prefix = [0];
  for (const x of nums) prefix.push(prefix[prefix.length - 1] + x);
  return prefix;
}

function subarraySum(nums, k) {
  const freq = new Map();
  freq.set(0, 1);
  let sum = 0;
  let count = 0;
  for (const x of nums) {
    sum += x;
    count += freq.get(sum - k) ?? 0;
    freq.set(sum, (freq.get(sum) ?? 0) + 1);
  }
  return count;
}
```

```
[1, 2, 3], k = 3
start {0: 1}
sum 1, ask for -2, miss, record 1
sum 3, ask for 0, hit 1, record 3     subarray [1, 2]
sum 6, ask for 3, hit 1, record 6     subarray [3]
count = 2

if you incremented first, a target of 0 would count the empty prefix as a new subarray
```

O(n) time expected. O(n) extra memory for the map. Building a prefix array for range queries is O(n) once, then O(1) per query.

The longest subarray with sum K stores the first index of each prefix, seeded as `0 → −1`. When you see `sum − k`, the length is `i − first.get(sum − k)`. You do not overwrite an earlier index, because an earlier start makes a longer subarray.

Zeros and ones, and a remainder

Equal number of 0s and 1s becomes a prefix sum if you map 0 to −1 and 1 to +1. A subarray with sum 0 is a balance. Seed index −1 at prefix 0, and store the first index only.

JavaScript's `%` keeps the sign of the dividend. `-1 % 5` is `-1`, not `4`. Two prefixes represent the same remainder only after you repair it:

THE ADDED K FOLDS A NEGATIVE REMAINDER BACK INTO 0..K−1.

```
function subarraysDivByK(nums, k) {
  const freq = new Map();
  freq.set(0, 1);
  let sum = 0;
  let count = 0;
  for (const x of nums) {
    sum += x;
    const mod = ((sum % k) + k) % k;
    count += freq.get(mod) ?? 0;
    freq.set(mod, (freq.get(mod) ?? 0) + 1);
  }
  return count;
}
```

```
(-7) % 5 in JavaScript is -2
((-7 % 5) + 5) % 5 = 3
that matches 3, because -7 and 3 differ by -10, a multiple of 5
```

Continuous subarray sum asks whether some subarray of length at least 2 has a sum divisible by k. Current constraints have k ≥ 1, so you will not divide by zero. Seed remainder 0 at index −1. Store only the first index of each remainder. A later copy of that remainder at index i is a hit when `i - first ≥ 2`. If you overwrite the first index, a long enough subarray shrinks to a short one and you miss it.

DO NOT UPDATE AN INDEX YOU HAVE ALREADY STORED.

```
function checkSubarraySum(nums, k) {
  const first = new Map();
  first.set(0, -1);
  let sum = 0;
  for (let i = 0; i < nums.length; i++) {
    sum += nums[i];
    const mod = ((sum % k) + k) % k;
    if (first.has(mod)) {
      if (i - first.get(mod) >= 2) return true;
    } else {
      first.set(mod, i);
    }
  }
  return false;
}
```

```
[23, 2, 4, 6, 7], k = 6
i=0 sum 23, mod 5, store 5 at 0
i=1 sum 25, mod 1, store 1 at 1
i=2 sum 29, mod 5, 2 - 0 = 2, return true
the subarray [2, 4] sums to 6
```

TRAPS

- Forgetting the seed 0 → 1. A subarray that starts at index 0 becomes invisible.
- Incrementing the frequency before the query. Target 0 then counts an empty piece.
- Using raw `%` on a negative sum. The buckets split, and equal remainders miss each other.
- Storing the latest index when you want the longest subarray, or when the length must be at least 2.
- Reaching for this map when every value is positive and you only need the shortest subarray with sum at least K. That is the chapter 7 window.

Practice

Stay before chapter 9. Then before you leave the chapter.

303. Range Sum Query - Immutable EASY STAY
Build prefix once in the constructor. sumRange(i, j) is prefix[j + 1] − prefix[i].

560. Subarray Sum Equals K MEDIUM STAY
Seed 0 → 1. Add the count of sum − k, then increment sum.

525. Contiguous Array MEDIUM STAY
Map 0 to −1. Longest prefix balance of 0. Seed index −1, store the first index.

974. Subarray Sums Divisible by K MEDIUM THEN
Frequency of the repaired remainder. The repair is ((sum % k) + k) % k.

523. Continuous Subarray Sum MEDIUM THEN
First index only, seed −1, length at least 2. k ≥ 1 under the current constraints.

REWRITE FROM MEMORY

Rewrite the frequency map, including the seed and the query-before-update, and the negative-modulo repair.

Carry forward. Chapter 9 runs this idea backward: instead of summing a range, you add a value onto a range in constant time.

CHAPTER 09 · BUILDS ON CHAPTER 8

Difference array

A range add is two writes. One scan turns those writes back into the array.

A prefix sum reads a range in constant time after a linear build. A difference array is the inverse. You want to add a value onto many ranges, and you will read the array only after every update is queued. Each range update becomes two writes. One later scan rebuilds the array.

RECOGNITION

- Many range additions, then one read of the final array. The premium problem Range Addition is this shape. You do not need it. The template below is the whole idea.
- Bookings, trips, or births, where each event has a start and an exclusive end.
- A small timeline, such as years from 1950 to 2050, or stops on a road.

SAY THIS BEFORE YOU CODE

The invariant: diff[i] is the change in the running value as I step onto index i. A range [L, R] inclusive writes +v at L and −v at R + 1.

THE EXTRA CELL HOLDS THE −V THAT WOULD HAVE LANDED JUST PAST THE END.

```
function applyRanges(n, updates) {
  const diff = new Array(n + 1).fill(0);
  for (const [L, R, v] of updates) {
    diff[L] += v;
    diff[R + 1] -= v;
  }
  const arr = new Array(n).fill(0);
  let run = 0;
  for (let i = 0; i < n; i++) {
    run += diff[i];
    arr[i] = run;
  }
  return arr;
}
```

```
n = 5, add 10 on [1, 3], add 5 on [2, 4]
diff starts [0, 0, 0, 0, 0, 0]
after [1, 3]: [0, 10, 0, 0, -10, 0]
after [2, 4]: [0, 10, 5, 0, -10, -5]
scan: 0, 10, 15, 15, 5
```

O(n + u) time for u updates. O(n) extra memory. Each update is O(1).

Corporate flight bookings are 1-based. Flight numbers run from 1 to n. A booking of `seats` on flights `first..last` inclusive is `diff[first - 1] += seats` and `diff[last] -= seats`. The subtraction lands on the first flight that should not receive the seats. When `last === n`, that write hits the extra cell and never enters the answer.

Car pooling uses a half-open ride. Passengers board at `from` and leave at `to`, so they do not occupy the destination. `diff[from] += passengers`, `diff[to] -= passengers`. Scan the road. If the running load ever exceeds capacity, return false. The timeline only needs to be as long as the farthest stop.

Maximum population year is the easy entry, and the timeline is tiny. A person is alive from `birth` inclusive to `death` exclusive. Shift the years by 1950 so the indices start at 0. When two years tie, keep the earlier one: update the best year only when the running population is strictly greater.

Second pass, still the same two writes

Maximum sum of any permutation scores each index by how many requests cover it. The difference array builds that coverage. Sort the values and the coverages, pair the largest value with the most requested index, and take the dot product modulo 1e9+7. Products of the constraints fit in a safe JavaScript integer if you reduce the sum modulo at every step.

Minimum number of K consecutive bit flips is a window flip stored as parity. If the current bit, after the flips already started, is 0, you must start a flip at i, which adds 1 to a difference that ends k steps later. If that end would fall past n, return −1. The running parity is the number of active flips modulo 2.

SECOND PASS. STARTING A FLIP IS A +1 THAT EXPIRES AT I + K.

```
function minKBitFlips(nums, k) {
  const n = nums.length;
  const diff = new Array(n + 1).fill(0);
  let cur = 0;
  let flips = 0;
  for (let i = 0; i < n; i++) {
    cur += diff[i];
    if ((nums[i] + cur) % 2 === 0) {
      if (i + k > n) return -1;
      flips += 1;
      cur += 1;
      diff[i + k] -= 1;
    }
  }
  return flips;
}
```

TRAPS

- Subtracting at R instead of R + 1 on an inclusive range. The last index never receives the value.
- Forgetting the 1-based shift on bookings. first is a flight number, not an index, until you subtract 1.
- Occupying the destination in car pooling. The passenger is gone at `to`.
- Updating the best year on a tie. The earliest year is the one you already stored.

Practice

Stay before chapter 10. The last two are a second pass.

1854. Maximum Population Year EASY STAY
Difference on years 1950..2050. Death is exclusive. Ties keep the earlier year.

1109. Corporate Flight Bookings MEDIUM STAY
1-based. diff[first − 1] += seats and diff[last] −= seats.

1094. Car Pooling MEDIUM STAY
Half-open trip [from, to). Fail as soon as the running load exceeds capacity.

1589. Maximum Sum Obtained of Any Permutation HARD SECOND PASS
Coverage by a difference array, sort both sides, dot product, modulo 1e9+7.

995. Minimum Number of K Consecutive Bit Flips HARD SECOND PASS
A flip is a parity update of length k. Return −1 when a needed flip runs off the end.

REWRITE FROM MEMORY
Rewrite the inclusive update +v at L and −v at R + 1, and the scan that rebuilds the array.

Carry forward. Chapter 10 builds a full left summary and a full right summary, then reads them together at each index.

CHAPTER 10 · BUILDS ON CHAPTERS 2 AND 5

A left pass and a right pass

Build the summary from the left, build it from the right, and combine them at i.

One running value was enough when the answer used only the past, or only the future. Some answers need both. Build a left-to-right array, build a right-to-left array, then combine them at i. The extra memory is O(n), and the passes are still linear.

RECOGNITION

- The value at i depends on everything to its left and everything to its right.
- Product except self, water above a bar, the length of a mountain, candies forced by both neighbors.
- You are not looking for the nearest greater element. You are looking at a summary of a whole side.

SAY THIS BEFORE YOU CODE

The invariant: after the left pass, left[i] describes indices strictly before i. After the right pass, right[i] describes indices strictly after i. i itself is applied when I combine them.

Product except self, no division

THE OUTPUT ARRAY STORES THE LEFT PRODUCTS, SO THE EXTRA MEMORY BEYOND THE ANSWER IS O(1).

```
function productExceptSelf(nums) {
  const n = nums.length;
  const out = new Array(n).fill(1);
  let left = 1;
  for (let i = 0; i < n; i++) {
    out[i] = left;
    left *= nums[i];
  }
  let right = 1;
  for (let i = n - 1; i >= 0; i--) {
    out[i] *= right;
    right *= nums[i];
  }
  return out;
}
```

```
[1, 2, 3, 4]
left pass writes [1, 1, 2, 6]
right = 1
i=3: 6 * 1 = 6, right = 4
i=2: 2 * 4 = 8, right = 12
i=1: 1 * 12 = 12, right = 24
i=0: 1 * 24 = 24
[24, 12, 8, 6]
```

O(n) time. O(1) extra memory besides the returned array.

Water, then the same decision with two indices

WATER AT I IS THE LOWER OF THE TWO SIDE-MAXES, MINUS THE BAR.

```
function trap(height) {
  const n = height.length;
  if (n === 0) return 0;
  const leftMax = new Array(n);
  const rightMax = new Array(n);
  leftMax[0] = height[0];
  for (let i = 1; i < n; i++) {
    leftMax[i] = Math.max(leftMax[i - 1], height[i]);
```

```
  }
  rightMax[n - 1] = height[n - 1];
  for (let i = n - 2; i >= 0; i--) {
    rightMax[i] = Math.max(rightMax[i + 1], height[i]);
  }
  let water = 0;
  for (let i = 0; i < n; i++) {
    const h = Math.min(leftMax[i], rightMax[i]) - height[i];
    if (h > 0) water += h;
  }
  return water;
}
```

```
[2, 0, 2]
leftMax  [2, 2, 2]
rightMax [2, 2, 2]
water    [0, 2, 0]
total 2
```

The O(1) version makes the same choice the container made. The side whose maximum is smaller is the bottleneck. Water on that side is already determined, so you can add it and move that index inward. You never need the other side's future once you know it is at least as tall.

MOVE THE SMALLER WALL. THAT SIDE'S WATER IS SETTLED.

```
function trap(height) {
  let l = 0;
  let r = height.length - 1;
  let leftMax = 0;
  let rightMax = 0;
  let water = 0;
  while (l < r) {
    if (height[l] < height[r]) {
      if (height[l] >= leftMax) leftMax = height[l];
      else water += leftMax - height[l];
      l += 1;
    } else {
      if (height[r] >= rightMax) rightMax = height[r];
      else water += rightMax - height[r];
      r -= 1;
    }
  }
  return water;
}
```

A mountain is an up-length plus a down-length

Walk left to right. `up[i]` is how many strict steps you climbed to reach i. Walk right to left. `down[i]` is how many strict steps you descend after i. A peak is an index where both are positive. Its mountain length is `up[i] + down[i] + 1`. A plateau has a zero on one side and does not count.

Candy is the second pass of the same shape. `left[i]` is 1, or `left[i - 1] + 1` when the rating rises. `right[i]` does that from the other side. The candies at i are the max of the two constraints, because the child must satisfy the higher neighbor on each side. Sum them.

TRAPS

- Multiplying the product-except-self by nums[i] before storing the left product. The cell would include itself.
- Forgetting that leftMax[i] includes height[i]. The water formula still works, because min(side, side) − height is 0 on a local peak.
- Calling a strictly increasing array a mountain. down[i] is 0 everywhere, so the best length stays 0.
- Giving candy by a single left-to-right pass. A descent forces extra candies that the left pass has not seen.

Practice

Stay before chapter 11. Candy is a second pass.

---

238. Product of Array Except Self **MEDIUM** **STAY**

Left products into the answer, right products on the way back. No division.

---

42. Trapping Rain Water **HARD** **STAY**

Write the two max arrays first. Then collapse them to the two-index version if you still have time.

---

845. Longest Mountain in Array **MEDIUM** **STAY**

up[i] and down[i] both positive. Length is up + down + 1. A flat peak is not a mountain.

---

135. Candy **HARD** **SECOND PASS**

Two passes. The answer at i is the max of the left constraint and the right constraint.

REWRITE FROM MEMORY

Rewrite product-except-self both directions, and say the water formula: min(leftMax, rightMax) − height.

Carry forward. Chapter 11 keeps only the best sum that ends at i, which is the prefix ending here minus the worst prefix before it.

Kadane

The best subarray ending here is nums[i], or the best ending before it plus nums[i]. Do not restart from 0.

You can build a prefix. The best subarray that ends at index j is `prefix[j + 1] − (the minimum prefix among prefix[0..j])`. Kadane keeps that quantity without storing the prefix. Let `bestEnding` be the best sum of a subarray that ends at the current index. It is either `nums[i]` alone, or `bestEnding + nums[i]` if the previous best ending helps. Start from `nums[0]`. Do not reset to 0.

RECOGNITION

- Maximum sum of a contiguous subarray. Empty is not allowed.
- Values may be negative. A window's shrink rule is the wrong tool, because you are not holding a constraint. You are discarding a negative prefix.
- Maximum product is the same recurrence plus the minimum product, because a negative multiplier flips them.
- A circular array is one ordinary Kadane, and one "total minus the minimum subarray," with the empty wrap rejected.

SAY THIS BEFORE YOU CODE

The invariant: bestEnding is the best non-empty subarray that ends exactly at i. best is the best of those, over every end I have seen.

START AT NUMS[0]. NEVER RESTART FROM 0.

```
function maxSubArray(nums) {
  let bestEnding = nums[0];
  let best = nums[0];
  for (let i = 1; i < nums.length; i++) {
    bestEnding = Math.max(nums[i], bestEnding + nums[i]);
    if (bestEnding > best) best = bestEnding;
  }
  return best;
}
```

```
[−2, −1]
bestEnding = −2, best = −2
i=1: max(−1, −2 + −1) = −1
best = −1
a reset to 0 would return 0, and 0 is the empty subarray, which is illegal

[1, −2, 3]
i=1: max(−2, 1 + −2) = −1, best stays 1
i=2: max(3, −1 + 3) = 3, best = 3
```

O(n) time. O(1) extra memory.

Product, with both ends saved before you overwrite

TEMPS, BECAUSE THE NEW MINIMUM NEEDS THE OLD MAXIMUM.

```
function maxProduct(nums) {
  let mx = nums[0];
  let mn = nums[0];
  let best = nums[0];
  for (let i = 1; i < nums.length; i++) {
    const x = nums[i];
    const nextMx = Math.max(x, mx * x, mn * x);
```

```
    const nextMn = Math.min(x, mx * x, mn * x);
    mx = nextMx;
    mn = nextMn;
    if (mx > best) best = mx;
  }
  return best;
}
```

```
[2, 3, -2, 4]
start mx 2, mn 2, best 2
x=3: mx 6, mn 3, best 6
x=-2: candidates  -2, 6*-2=-12, 3*-2=-6
      mx = -2, mn = -12, best stays 6
x=4: candidates 4, -2*4=-8, -12*4=-48
      mx = 4, mn = -48, best stays 6
the flip at -2 used the old mn and the old mx together
```

Circular: reject the empty wrap

A circular subarray is either ordinary, or it wraps. The wrap is the total minus a non-wrapping middle that you throw away. The middle you throw away is the minimum subarray. If every value is negative, the minimum subarray is the whole array, the wrap is empty, and `total - minSubarray` is 0. Return the ordinary maximum instead. The test is `maxBest < 0`.

IF THE BEST ORDINARY SUM IS NEGATIVE, EVERY WRAP IS WORSE OR EMPTY.

```
function maxSubarraySumCircular(nums) {
  let maxEnding = nums[0];
  let maxBest = nums[0];
  let minEnding = nums[0];
  let minBest = nums[0];
  let total = nums[0];
  for (let i = 1; i < nums.length; i++) {
    const x = nums[i];
    maxEnding = Math.max(x, maxEnding + x);
    if (maxEnding > maxBest) maxBest = maxEnding;
    minEnding = Math.min(x, minEnding + x);
    if (minEnding < minBest) minBest = minEnding;
    total += x;
  }
  if (maxBest < 0) return maxBest;
  return Math.max(maxBest, total - minBest);
}
```

```
[5, -3, 5]
ordinary best = 7? let's compute:
maxEnding: 5, then max(-3, 2) = 2, then max(5, 7) = 7. maxBest = 7
minEnding: 5, then min(-3, 2) = -3, then min(5, 2) = 2. minBest = -3
total = 7
7 is not negative, so max(7, 7 - (-3)) = 10
the wrap is the two 5s

[-3, -2, -1]
maxBest = -1
minBest = -6, total = -6, total - minBest = 0
maxBest < 0, return -1, not the empty wrap
```

Every rise is a transaction

Best time to buy and sell stock II allows as many transactions as you want, with no overlap. The sum of every positive difference `prices[i] - prices[i - 1]` is the profit. You can prove it by noticing that any climb from a valley to a peak equals the sum of the daily rises along it.

Stock III (at most two transactions), cooldown, and the transaction fee are states, not Kadane. Link them and leave them for later: 123, 309, and 714. A state is "holding or not," plus how many buys you have used, plus whether yesterday was a sale. Do not force those into `bestEnding`.

TRAPS

- Resetting a negative bestEnding to 0. All-negative input returns 0.
- Writing `mx = Math.max(x, mx * x, mn * x)` and then computing mn from the new mx. The flip is gone.
- Returning `total - minBest` on an all-negative circular array. That expression is 0, the empty wrap.
- Using Kadane for "maximum sum with length at least K" without care, or for a sum bounded by K. A bound is a window when values are non-negative, and a deque of prefixes when they are not. Chapter 15 has the deque.

Practice

Stay before chapter 12. The circular problem is Then. The three state problems are later, on a second pass.

53. Maximum Subarray MEDIUM STAY
bestEnding is nums[i] or bestEnding + nums[i]. Start at nums[0].

152. Maximum Product Subarray MEDIUM STAY
Track min and max. Compute both temps from the old pair, then assign.

122. Best Time to Buy and Sell Stock II MEDIUM STAY
Add every positive daily rise. Unlimited non-overlapping transactions.

918. Maximum Sum Circular Subarray MEDIUM THEN
max(ordinary, total − min subarray). If ordinary is negative, return it.

123. Best Time to Buy and Sell Stock III HARD SECOND PASS
At most two transactions. This is a state machine, not Kadane.

309. Best Time to Buy and Sell Stock with Cooldown MEDIUM SECOND PASS
After a sale you must skip a day. States: hold, sold, rest.

714. Best Time to Buy and Sell Stock with Transaction Fee MEDIUM SECOND PASS
A fee on each sale. Hold versus cash, and the fee is paid when you sell.

REWRITE FROM MEMORY
Rewrite Kadane on [-2, -1], the product temps, and the circular rejection of the empty wrap.

Carry forward. Chapter 12 pays n log n for an order, then runs the chapter 5 scan on pairs and on intervals.

CHAPTER 12 · BUILDS ON CHAPTERS 4 AND 5

Sort, then scan

Pay n log n to create an order. Then a two-end scan, with duplicate skips, finishes the job.

A map answered "have I seen this value?" A sort answers a different question: put the values in an order so that a chapter 5 scan can discard an end. You pay O(n log n) for the order. After that, the scan is linear, or n^2 for 3Sum, which is the accepted cost.

RECOGNITION

- Pairs or triples that sum to a target, and a sort is allowed.
- Intervals: merge, insert, erase overlaps, shoot arrows, intersect two lists.
- You pair the lightest person with the heaviest, or you fix the largest side of a triangle.
- The array is already sorted and you only scan runs. Summary ranges is that scan, with no sort of your own.

SAY THIS BEFORE YOU CODE

I will sort, name the order I just created, and then say which duplicate skip keeps the answer unique.

3Sum

Sort. For each index i, run two-sum on the suffix with target `-nums[i]`. Skip a repeated i. After a hit, skip a repeated l and a repeated r, or the same triple is emitted again. If `nums[i] > 0`, every later sum is positive, so you can break. That break belongs to target 0. Do not copy it onto a 4Sum whose target can be negative.

SKIP DUPLICATE I, AND SKIP BOTH ENDS AFTER A HIT.

```
function threeSum(nums) {
  nums.sort((a, b) => a - b);
  const out = [];
  for (let i = 0; i < nums.length; i++) {
    if (nums[i] > 0) break;
    if (i > 0 && nums[i] === nums[i - 1]) continue;
    let l = i + 1;
    let r = nums.length - 1;
    while (l < r) {
      const sum = nums[i] + nums[l] + nums[r];
      if (sum === 0) {
        out.push([nums[i], nums[l], nums[r]]);
        l += 1;
        r -= 1;
        while (l < r && nums[l] === nums[l - 1]) l += 1;
        while (l < r && nums[r] === nums[r + 1]) r -= 1;
      } else if (sum < 0) {
        l += 1;
      } else {
        r -= 1;
      }
    }
  }
  return out;
}
```

```
[-1, 0, 1, 2, -1, -4]
sorted [-4, -1, -1, 0, 1, 2]
i = -4: no pair in the suffix sums to 4
i = -1 at index 1
  l = -1, r = 2, sum 0 → emit [-1, -1, 2]
  l and r move to 0 and 1, sum 0 → emit [-1, 0, 1]
i = -1 at index 2, same as previous i, skip
i = 0: the suffix is positive, sums stay positive
i > 0 would break
answer [[-1, -1, 2], [-1, 0, 1]]
```

O(n²) time after the O(n log n) sort. O(1) extra memory besides the output, if the sort is in place.

3Sum closest keeps the sum with the smallest absolute distance instead of collecting hits. 4Sum adds one more index, with the same duplicate skips. You may prune an i when the smallest possible four-sum from here already exceeds the target, and skip an i when the largest possible four-sum is still too small. Those prunes are optional. The duplicate skips are not.

Lightest with heaviest, and triangles

Boats: sort the people. While `l <= r`, the heaviest remaining person takes a boat. The lightest joins them when the two fit. One person, the case `l === r`, still takes one boat. The problem promises nobody is heavier than the limit.

A triangle needs `a + b > c` for the largest side c. Sort, fix the largest side at k, and set l and r inside the prefix. If `nums[l] + nums[r] > nums[k]`, then every index from l through r − 1 also works with this r, because those values are at least `nums[l]`. Add `r - l`, then move r left. Otherwise move l right.

Intervals

Sort closed intervals by start. They overlap when the next start is <= the last end you are holding. A touch, `start === lastEnd`, is an overlap for merge. Extend the end when the new interval sticks out. Meeting Rooms, the premium cousin, is this sort plus one question: after sorting, does any start fall before the previous end? You do not need that problem. The sentence is the whole variant.

OVERLAP TEST FOR CLOSED INTERVALS: START <= LAST END. THE CODE USES THE COMPLEMENTARY TEST, START > LAST END, TO OPEN A NEW INTERVAL.

```
function merge(intervals) {
  intervals.sort((a, b) => a[0] - b[0]);
  const out = [];
  for (const [start, end] of intervals) {
    const last = out[out.length - 1];
    if (out.length === 0 || start > last[1]) out.push([start, end]);
    else if (end > last[1]) last[1] = end;
  }
  return out;
}
```

```
[[1, 3], [2, 6], [8, 10], [15, 18]]
already sorted
[1, 3] starts the output
2 <= 3, extend end to 6
8 > 6, push [8, 10]
15 > 10, push [15, 18]
[[1, 6], [8, 10], [15, 18]]
```

Insert interval has a one-pass version you should be able to say first: push the new interval into the list, sort, and reuse merge. The linear version does not sort. Copy every interval that ends before the new one starts. Then fold every interval that overlaps the new one into a single `[start, end]`. Push that, and copy the tail.

COPY, FOLD, COPY. TOUCHING MERGES, SO THE TESTS ARE STRICT ON THE LEFT AND CLOSED ON THE FOLD.

```
function insert(intervals, newInterval) {
  const out = [];
  let [start, end] = newInterval;
  let i = 0;
  while (i < intervals.length && intervals[i][1] < start) {
    out.push(intervals[i]);
    i += 1;
  }
  while (i < intervals.length && intervals[i][0] <= end) {
    start = Math.min(start, intervals[i][0]);
    end = Math.max(end, intervals[i][1]);
    i += 1;
  }
  out.push([start, end]);
  while (i < intervals.length) {
    out.push(intervals[i]);
    i += 1;
  }
  return out;
}
```

Non-overlapping intervals: sort by end, not by start. Keep an interval when its start is >= the end you last kept. Touching is allowed, because `[1, 2]` and `[2, 3]` do not overlap in the sense this problem uses. The answer is how many you erase, which is `n - kept`.

Arrows burst balloons: sort by end again. A new arrow is required only when the next balloon starts strictly after the current arrow's end. Touching balloons share an arrow. The inequality is the whole difference between this problem and the previous one.

Two sorted interval lists: one index on each. The overlap of the two current intervals, if any, runs from the max of the starts to the min of the ends. Advance the list whose current interval ends first.

Summary ranges does not sort. The input is already sorted and unique. Scan, and whenever the next value is not exactly one more, close the range you were building.

TRAPS

- Sorting with the default comparator. [10, 2] as interval starts will not order numerically.
- Skipping only the i duplicate in 3Sum. The l and r duplicates reprint the same triple.
- Breaking 4Sum because `nums[i] > 0`. That break is for a target of 0.
- Merging on `start < lastEnd` and leaving a touch unmerged. Closed intervals touch at the endpoint.
- Sorting non-overlapping intervals by start and then being greedy. The earliest ending interval is the one that leaves the most room. Sort by end.
- Using >= for a new arrow. Touching balloons share one.

Practice

Stay before chapter 13. Then before you leave the chapter.

15. 3Sum MEDIUM STAY
Sort, skip duplicate i, skip both ends after a hit, break once nums[i] > 0.

56. Merge Intervals MEDIUM STAY
Sort by start. A new piece starts only when start > last end.

881. Boats to Save People MEDIUM STAY
Sort. Pair lightest with heaviest while l <= r. Always send the heavy person.

228. Summary Ranges EASY STAY

Already sorted. Close a range when the next value is not current + 1.

16. 3Sum Closest MEDIUM THEN

The same loops. Keep the sum closest to the target instead of the zeros.

18. 4Sum MEDIUM THEN

One more loop, the same duplicate skips. Do not break just because a value is positive.

611. Valid Triangle Number MEDIUM THEN

Fix the largest side. When l and r fit, count r − l and move r.

57. Insert Interval MEDIUM THEN

Say "push and reuse merge" first. Then write the copy, fold, copy pass.

435. Non-overlapping Intervals MEDIUM THEN

Sort by end. Keep a start that is >= the last kept end. Touching is allowed.

452. Minimum Number of Arrows to Burst Balloons MEDIUM THEN

Sort by end. A new arrow only when start > current end. Touching shares.

986. Interval List Intersections MEDIUM THEN

Two indices. Overlap is [max starts, min ends]. Advance the interval that ends first.

REWRITE FROM MEMORY

Rewrite 3Sum, including both duplicate skips, and the merge test start > last end.

Carry forward. Chapter 13 does not sort. The value itself tells you the index where it belongs.

CHAPTER 13 · BUILDS ON CHAPTERS 6 AND 12

Place each value where it belongs

A value v in 1..n wants index v − 1. Swap it home. Do not advance past an unseen swap.

Sorting ordered the values. Here the legal values are already indices. A value `v` in `1..n` wants index `v - 1`. A value in `0..n-1` wants index `v`. You swap until each value sits at home, or until you discover it cannot, because another copy already lives there.

RECOGNITION

- Values are in 1..n or 0..n, and the question is a missing number, a duplicate, or a first positive.
- Three-way partition: zeros, ones, and twos.
- You may modify the array. If the problem forbids modification, the duplicate becomes a cycle in the index map i → nums[i].
- One value appears once and every other value appears twice or three times. XOR, or a bit count, replaces the placement.

SAY THIS BEFORE YOU CODE

The invariant: values already placed at their home index stay there. A swap moves some value onto its home, or I have found a duplicate.

Three regions

Dutch national flag uses three indices. `[0, lo)` holds 0. `[lo, mid)` holds 1. `(hi, n)` holds 2. `[mid, hi]` is unseen. Read `nums[mid]`. A 0 swaps with `lo` and both `lo` and `mid` advance, because the value that arrived from `lo` is a 1 you have already accepted, or it is `mid` itself. A 1 just advances `mid`. A 2 swaps with `hi` and `hi` moves left. Do not advance `mid` after that swap. The value that arrived from `hi` has not been read.

AFTER A SWAP WITH HI, MID STAYS. THE NEW NUMS[MID] IS UNPROCESSED.

```
function sortColors(nums) {
  let lo = 0;
  let mid = 0;
  let hi = nums.length - 1;
  while (mid <= hi) {
    if (nums[mid] === 0) {
      const tmp = nums[lo];
      nums[lo] = nums[mid];
      nums[mid] = tmp;
      lo += 1;
      mid += 1;
    } else if (nums[mid] === 1) {
      mid += 1;
    } else {
      const tmp = nums[mid];
      nums[mid] = nums[hi];
      nums[hi] = tmp;
      hi -= 1;
    }
  }
  return nums;
}
```

```
[2, 0, 1]
mid=0 sees 2, swap with hi → [1, 0, 2], hi = 1, mid stays 0
mid=0 sees 1, mid = 1
mid=1 sees 0, swap with lo → [0, 1, 2], lo = 1, mid = 2
mid > hi, stop
if you had advanced mid after the first swap, you would skip the 1
```

O(n) time. O(1) extra memory.

Cyclic placement

IGNORE VALUES OUTSIDE 1..N. V WANTS INDEX V − 1.

```
function firstMissingPositive(nums) {
  const n = nums.length;
  let i = 0;
  while (i < n) {
    const v = nums[i];
    const dest = v - 1;
    if (v >= 1 && v <= n && nums[dest] !== v) {
      const tmp = nums[i];
      nums[i] = nums[dest];
      nums[dest] = tmp;
    } else {
      i += 1;
    }
  }
  for (let j = 0; j < n; j++) {
    if (nums[j] !== j + 1) return j + 1;
  }
  return n + 1;
}
```

```
[3, 4, -1, 1]
3 belongs at index 2: [-1, 4, 3, 1]
-1 is outside 1..4, skip
4 belongs at index 3: [-1, 1, 3, 4]
1 belongs at index 0: [1, -1, 3, 4]
scan: index 1 holds -1, not 2, so the answer is 2
```

Missing number is the same placement for values in `0..n` living in an array of length n, so one value is absent and the hole is the answer. You can also XOR every index with every value, or subtract the sum from `n * (n + 1) / 2`. The sum fits in a safe integer for the usual constraints. Mention all three. Write the one you can finish.

Find the duplicate, problem 287, says you must not modify the array. Treat the array as the function `i → nums[i]`. Values are in `1..n` with length `n + 1`, so the walk from index 0 enters a cycle, and the entrance of the cycle is the duplicate. Floyd's two indices find it: move slow by one step and fast by two until they meet, then reset one index to `nums[0]` and move both one step at a time. If modification is allowed, the cyclic swap above stops when `nums[dest] === v` and `dest !== i`, and that `v` is the duplicate.

287, NO WRITES. THE CYCLE ENTRANCE IS THE REPEATED VALUE.

```
function findDuplicate(nums) {
  let slow = nums[0];
  let fast = nums[0];
  do {
    slow = nums[slow];
    fast = nums[nums[fast]];
  } while (slow !== fast);
  slow = nums[0];
  while (slow !== fast) {
    slow = nums[slow];
    fast = nums[fast];
  }
```

```
  return slow;
}
```

Marks in the sign bit

When every value is in `1..n` and you may modify, the sign of `nums[v - 1]` can record that you have seen `v`. Read `Math.abs` before you use the value as an index. Disappeared numbers are the indices whose cell stayed positive. All duplicates are the values whose cell was already negative. Set mismatch returns both: the value that finds a negative cell is the duplicate, and the index that stayed positive is the missing number.

One number alone

XOR is its own inverse. `x ^ x` is 0, and `x ^ 0` is `x`. XOR every element of an array where every value except one appears twice. The pairs vanish. The singleton remains. Order does not matter.

Single number II: every value except one appears three times. Keep two bit masks. `ones` holds bits seen 3t+1 times, `twos` holds bits seen 3t+2 times. Seeing a bit again moves it from ones to twos, then off both, which is the multiple of three. The update uses the other mask to clear a bit that has moved on:

SECOND PASS. JAVASCRIPT BITWISE OPS ARE 32-BIT SIGNED. THESE INPUTS FIT.

```
function singleNumber(nums) {
  let ones = 0;
  let twos = 0;
  for (const x of nums) {
    ones = (ones ^ x) & ~twos;
    twos = (twos ^ x) & ~ones;
  }
  return ones;
}
```

Single number III: two values appear once, everything else twice. XOR the whole array. The result is `a ^ b`, which is nonzero, so some bit differs between a and b. Take the lowest set bit, `xor & -xor` in 32-bit arithmetic. Split the numbers into two groups by that bit, and XOR inside each group. Each group reduces to one of the singletons.

TRAPS

- Incrementing mid after the swap with hi. The swapped-in value is unseen.
- Swapping in a cycle when `nums[dest] === v` already. You spin forever. The inequality is the stop, and it is also how you notice a duplicate.
- Using a raw negative cell as an index. Take the absolute value first.
- Running Floyd when the problem allows writes and expects the cyclic solution, or writing into the array when 287 forbids it. Read the constraint.
- Treating XOR as a sum. It cancels pairs. It does not cancel triples. Triples need the ones/twos masks or a count of bits modulo 3.

Practice

Stay before chapter 14. Then before you leave the chapter. The two bit problems are a second pass.

75. Sort Colors MEDIUM STAY

Three regions. Do not advance mid after swapping with hi.

268. Missing Number EASY STAY

Cyclic placement, or the sum formula, or XOR of indices and values.

136. Single Number EASY STAY

XOR everything. Pairs cancel.

448. Find All Numbers Disappeared in an Array EASY STAY

Negate nums[abs(x) − 1]. Positive cells at the end are the missing numbers.

442. Find All Duplicates in an Array MEDIUM THEN

The same marks. A cell that is already negative means the value is a duplicate.

645. Set Mismatch EASY THEN

One pass finds the duplicate by a second visit. The positive cell is the missing value.

287. Find the Duplicate Number MEDIUM THEN

Do not modify. Floyd on i → nums[i]. The entrance is the duplicate.

41. First Missing Positive HARD THEN

Place v at v − 1 when v is inside 1..n. The first mismatch is the answer.

137. Single Number II MEDIUM SECOND PASS

ones and twos. A bit seen three times clears out of both masks.

260. Single Number III MEDIUM SECOND PASS

XOR all, split by one set bit of that XOR, XOR inside each group.

REWRITE FROM MEMORY

Redraw the three Dutch-flag regions and rewrite the loop until the hi-swap leaves mid alone.

Carry forward. Chapter 14 searches a sorted index range, and then searches the answer itself, with one loop shape.

Binary search

One loop: while lo < hi, if ok(mid) then hi = mid, else lo = mid + 1.

You can put a value at its index. Binary search is the other use of a sorted range: throw away half because a predicate says the answer cannot live there. This book has one loop. The range is half-open, `[lo, hi)`. `hi` is not a candidate. The loop runs while `lo < hi`. The mid calculation rounds down, so `mid` is always strictly less than `hi` while the range is non-empty. Setting `hi = mid` shrinks the range. Setting `lo = mid + 1` shrinks it too.

RECOGNITION

- A sorted array, and you want the first index where a predicate becomes true.
- Search insert, first occurrence, first bad version, integer square root: all of them are that lower bound.
- The thing you are searching is not an index. It is a speed, a capacity, a day, or a largest sum. If "can I achieve X?" is false for every value below some boundary and true above it, you can binary-search X.
- A rotated sorted array still has one sorted half. A peak tells you which neighbor is higher.

SAY THIS BEFORE YOU CODE

The invariant: the answer is inside [lo, hi). If ok(mid), the answer is at mid or to its left, so hi = mid. Otherwise the answer is strictly to the right, so lo = mid + 1.

THE ONLY TEMPLATE. OK(MID) MEANS "MID IS ALREADY LARGE ENOUGH."

```
function lowerBound(nums, target) {
  // [lo, hi). First index with nums[i] >= target, or nums.length.
  let lo = 0;
  let hi = nums.length;
  while (lo < hi) {
    const mid = lo + Math.floor((hi - lo) / 2);
    if (nums[mid] >= target) hi = mid;
    else lo = mid + 1;
  }
  return lo;
}
```

```
nums = [1, 3, 3, 5], target 3
[0, 4) mid 2, nums[2] = 3 >= 3, hi = 2
[0, 2) mid 1, nums[1] = 3 >= 3, hi = 1
[0, 1) mid 0, nums[0] = 1 < 3, lo = 1
lo === hi === 1, the first 3

the infinite loop, so you never write it:
lo = 0, hi = 1, mid = 0
if the failing branch sets lo = mid, lo stays 0 forever
the failing branch must be lo = mid + 1
```

O(log n) comparisons. The feasibility test wrapped around this loop can make the whole algorithm O(n log A), where A is the size of the answer range.

Search insert position is `lowerBound`. The first and last position of a target are `lowerBound(target)` and `lowerBound(target + 1) − 1`. If the first index is past the end, or does not hold the target, the answer is `[−1, −1]`. First bad version is `lowerBound` on versions `1..n` with `hi = n + 1` and `ok = isBadVersion`.

Integer square root: search `[0, x + 1)` for the first integer whose square is greater than x, then subtract 1. For the constraint `x <= 2^31 − 1`, `mid * mid` is a safe JavaScript number. If you ever search a larger domain, compare with `mid > x / mid` instead of multiplying.

The same loop, four stories

Koko eats at speed `s` and needs `ceil(pile / s)` hours on a pile. You want the smallest s that finishes within h. The low end is 1. The high end, exclusive, is one more than the largest pile. `ok(s)` sums the hours.

Ship capacity: low is the heaviest box, because the ship must carry it. High, exclusive, is one more than the total weight. `ok(cap)` counts how many days the capacity needs, by filling a ship until the next box does not fit.

Bloom day: if `m * k` exceeds the number of flowers, return −1. Otherwise search the day. A day can make a bouquet when k adjacent flowers have all bloomed by that day. Reset the adjacent run when a flower is late.

Split array largest sum is the ship again. Low is the largest element. High, exclusive, is one more than the total. `ok(cap)` is "I can cover the array in at most m contiguous parts, each summing to at most cap." It is a second-pass problem, and it is the same function.

One half of a rotation is sorted

Assume the values are unique. At mid, compare `nums[lo]` with `nums[mid]`. If the left half `[lo, mid]` is sorted, the target is inside it only when it sits between `nums[lo]` and `nums[mid]`. Otherwise it sits in the other half. The right half is the sorted one in the other branch. Use `nums[hi − 1]` as the rightmost included value, because `hi` is exclusive.

UNIQUE VALUES. THE HALF YOU JUST TESTED IS THE SORTED ONE.

```
function search(nums, target) {
  let lo = 0;
  let hi = nums.length;
  while (lo < hi) {
    const mid = lo + Math.floor((hi − lo) / 2);
    if (nums[mid] === target) return mid;
    if (nums[lo] <= nums[mid]) {
      if (nums[lo] <= target && target < nums[mid]) hi = mid;
      else lo = mid + 1;
    } else if (nums[mid] < target && target <= nums[hi − 1]) {
      lo = mid + 1;
    } else {
      hi = mid;
    }
  }
  return −1;
}
```

```
[4, 5, 6, 7, 0, 1, 2], target 0
[0, 7) mid 3 value 7, left half [4, 7] is sorted, 0 is not in it, lo = 4
[4, 7) mid 5 value 1, left half [0, 1] is sorted, 0 is in it, hi = 5
[4, 5) mid 4 value 0, return 4
```

The minimum of a rotated array is the only place the sequence drops. While `lo < hi` on a closed right end, if `nums[mid] > nums[hi]` the minimum is strictly right of mid. Otherwise mid might be the minimum, so `hi = mid`. The range still shrinks, because mid is computed below hi.

Rotated array II may contain duplicates. When `nums[lo]`, `nums[mid]`, and `nums[hi]` are equal, you cannot tell which half is sorted. Shrink one end by one, `lo += 1`, and continue. The worst case is O(n), on an array of identical values with the target missing or sitting at the end.

Peaks, pairs, and a window of k

A peak is an index greater than or equal to its neighbors. The ends are peaks if they beat their one neighbor, because the missing neighbor counts as smaller. If `nums[mid] > nums[mid + 1]`, a peak exists at mid or to its left. Otherwise a peak exists to the right. Move toward the higher neighbor.

A single element in a sorted array sits between pairs. Before that element, pairs occupy `(even, odd)`. If `nums[even] === nums[even + 1]`, the single element is to the right, and you set `lo = even + 2`.

Otherwise it is at this even index or to the left.

K closest elements: binary-search the start of a window of length k. The legal starts are `0 .. n − k`. Prefer the window that starts at mid over the one that starts at mid + 1 when `x − arr[mid]` is less than or equal to `arr[mid + k] − x`. An equal distance keeps the left window, which holds the smaller values.

Second pass

Median of two sorted arrays: binary-search the cut on the shorter array. Take `i` elements from it and `half − i` from the longer one, where `half` is `floor((m + n + 1) / 2)`. The cut is correct when the left side of each array is <= the right side of the other. If you took too much from the shorter array, move the cut left. An odd total returns the max of the two left edges. An even total averages that with the min of the two right edges.

Find in a mountain array: find the peak with the neighbor test, using the class's `get`. Binary-search the ascending side for the target. If it is absent, binary-search the descending side, and remember that "too small" on a descent means move left.

TRAPS

- Setting `lo = mid` on the failing branch. When two indices remain, mid equals lo, and the loop never ends.
- Mixing inclusive `hi` with the half-open update `hi = mid` without re-reading the invariant. Pick one and comment it.
- Searching a rotated array as if the whole array were sorted. Name which half is sorted before you discard anything.
- Forgetting that rotated II can be linear. Say the worst case out loud.
- Returning the square-root boundary without subtracting 1, if your predicate was "square is already too big."

Practice

Stay before chapter 15. Then before you leave the chapter. Three problems are a second pass.

704. Binary Search EASY STAY
The template, then check that nums[lo] equals the target. The classic version returns the index or −1.

35. Search Insert Position EASY STAY
lowerBound itself. The insertion index may equal nums.length.

34. Find First and Last Position of Element in Sorted Array MEDIUM STAY
lower(target) and lower(target + 1) − 1. Absent target returns [-1, -1].

875. Koko Eating Bananas MEDIUM STAY
Search the speed. ok sums ceil(pile / speed) and compares with h.

33. Search in Rotated Sorted Array MEDIUM STAY
Unique values. One half is sorted. Discard the half that cannot hold the target.

153. Find Minimum in Rotated Sorted Array MEDIUM STAY
If nums[mid] > nums[hi], the min is to the right. Otherwise hi = mid.

278. First Bad Version EASY THEN
lowerBound on 1..n with hi = n + 1. ok is isBadVersion. Call it as rarely as the loop does.

69. Sqrt(x) EASY THEN
First integer whose square exceeds x, then subtract 1.

367. Valid Perfect Square EASY THEN
The integer square root, squared, equals the input. 0 and 1 are squares.

1011. Capacity To Ship Packages Within D Days MEDIUM THEN
Low is the heaviest package. ok counts the days a capacity needs.

1482. Minimum Number of Days to Make m Bouquets MEDIUM THEN
Adjacent flowers. Return −1 when m * k > n.

81. Search in Rotated Sorted Array II MEDIUM THEN
When both ends equal mid, shrink an end. Worst case O(n).

162. Find Peak Element MEDIUM THEN
Move toward the higher neighbor. An end can be the peak.

852. Peak Index in a Mountain Array MEDIUM THEN
The same neighbor test. A mountain has one peak, so the answer is unique.

540. Single Element in a Sorted Array MEDIUM THEN
Force mid even. Equal pair means the single element is to the right.

658. Find K Closest Elements MEDIUM THEN
Binary-search the window start. A distance tie keeps the smaller values.

410. Split Array Largest Sum HARD SECOND PASS
Same predicate as the ship. Minimize the largest part sum, using at most m parts.

1095. Find in Mountain Array HARD SECOND PASS
Peak, then ascending search, then descending search. Use get, and cache nothing the API forbids.

4. Median of Two Sorted Arrays HARD SECOND PASS
Partition the shorter array. Left edges <= right edges. Odd and even totals differ.

REWRITE FROM MEMORY
Rewrite lowerBound, and say the infinite loop that lo = mid creates when the range stops shrinking.

Carry forward. Chapter 15 keeps a stack of indices in a forced order, so the next greater element is the value that pops you.

Monotonic stack

Pop while the top is smaller. The stack of indices stays decreasing from bottom to top.

A right-to-left pass gave you the greatest element anywhere on the right. The next greater element is a nearer, weaker question, and a running maximum cannot answer it. Keep a stack of indices whose values are decreasing from bottom to top. When the new value is larger than the top, the top's next greater element is this value. Pop it. The stack stays decreasing, so each index is pushed once and popped once.

RECOGNITION

- Next greater, next smaller, previous smaller, or the distance until a warmer day.
- A span: how many values to the left are <= today, answered by collapsing them into one stack entry.
- A histogram bar as the shortest bar of a rectangle. Sentinels on both ends force every bar to be popped.
- The maximum in a sliding window. A deque, not a variable.
- A sum with negatives, and you want the shortest subarray at least K. A deque of prefix indices, increasing by prefix value.

SAY THIS BEFORE YOU CODE

The invariant: the stack is decreasing from bottom to top. Everything below an index is greater than it and still waiting for its next greater element. Everything I have popped already has an answer.

POP WHILE THE TOP IS STRICTLY SMALLER. THE STACK STORES INDICES.

```
function nextGreater(nums) {
  const n = nums.length;
  const ans = new Array(n).fill(-1);
  const stack = [];
  for (let i = 0; i < n; i++) {
    while (stack.length > 0 && nums[stack[stack.length - 1]] < nums[i]) {
      ans[stack.pop()] = nums[i];
    }
    stack.push(i);
  }
  return ans;
}
```

```
[2, 1, 2, 4]
i=0 push 0
i=1 value 1, top 2 is not smaller, push 1
i=2 value 2 pops index 1, ans[1] = 2, top 2 is not smaller, push 2
i=3 value 4 pops index 2, ans[2] = 4, pops index 0, ans[0] = 4
ans = [4, 2, 4, -1]
```

O(n) time. Each index is pushed once and popped once. O(n) extra memory.

Daily temperatures stores the same indices and writes `i - j`, the distance, instead of the temperature. A circular array scans `i` from 0 to `2n - 1` and reads `nums[i % n]`. Push an index only when `i < n`, so each position enters the stack once. Write answers only for those real indices. Online stock span stores pairs `[price, span]`. Pop while the top price is <= today's price, and add the popped spans. Today includes itself, so the span starts at 1.

The window maximum

INDICES, DECREASING VALUES. A HEAD POINTER REPLACES ARRAY.SHIFT.

```
function maxSlidingWindow(nums, k) {
  const dq = [];
```

```
  let head = 0;
  const out = [];
  for (let i = 0; i < nums.length; i++) {
    while (head < dq.length && dq[head] <= i - k) head += 1;
    while (head < dq.length && nums[dq[dq.length - 1]] <= nums[i]) dq.pop();
    dq.push(i);
    if (i >= k - 1) out.push(nums[dq[head]]);
  }
  return out;
}
```

`Array.shift` moves every element. A head index makes the front pop O(1). Drop a head that has fallen out of the window. Pop from the back while the back value is smaller than or equal to the new value, because the new value is later and at least as large, so the back can never be the maximum again.

Histogram, with a zero on each end

THE SENTINELS MAKE EVERY BAR POP, AND THE STACK IS NEVER EMPTY WHEN YOU READ THE LEFT EDGE.

```
function largestRectangleArea(heights) {
  const h = [0, ...heights, 0];
  const stack = [0];
  let best = 0;
  for (let i = 1; i < h.length; i++) {
    while (h[stack[stack.length - 1]] > h[i]) {
      const mid = stack.pop();
      const left = stack[stack.length - 1];
      const width = i - left - 1;
      const area = h[mid] * width;
      if (area > best) best = area;
    }
    stack.push(i);
  }
  return best;
}
```

```
heights [2, 1, 2] become [0, 2, 1, 2, 0]
i=1 value 2, push
i=2 value 1 pops the 2: left is the leading 0, width 1, area 2
i=3 value 2, push
i=4 value 0 pops the 2: width 1, area 2
         pops the 1: left is the leading 0, width 3, area 3
best 3
```

The bar you pop is the shortest bar of a rectangle that extends from the new top, exclusive, to `i`, exclusive. The width is `i - left - 1`.

Second pass: sums, contributions, and 132

Shortest subarray with sum at least K, when negatives are allowed, builds the prefix of chapter 8 and keeps a deque of prefix indices whose prefix values increase. A later, smaller prefix makes an earlier prefix useless as a start, because it is both larger and older. When `prefix[i] - prefix[head] >= k`, that head is a candidate start. Later ends would only make a longer subarray for the same start, so you pop the head. The answer is the minimum length, or −1.

Absolute difference within a limit uses two deques, one decreasing for the max and one increasing for the min, with the same head-pointer trick. Shrink `l` while `max - min` exceeds the limit, and drop a head that equals `l`.

Sum of subarray minimums asks, for each index, how many subarrays have that index as their minimum. Count the distance to the previous strictly smaller bar, and the distance to the next smaller-or-equal bar. Strict on one side and non-strict on the other means equal minimums are given to the left copy, so they are not counted twice. The contribution is `value * left * right`, summed modulo 1e9+7.

132 pattern, scanned from the right: the stack holds possible "3" values, decreasing. Pop a smaller top into a variable `ak`, the best "2" you have seen, which had a larger value to its left. When the current value is smaller than `ak`, that current value is a "1" further left, and you return true.

TRAPS

- Storing values when you needed the distance. Store the index, and read the value from the array.
- Using `Array.shift` on the deque. The window maximum becomes O(nk).
- Popping the histogram with `>=` or `>` at random. Strict `>` against the new bar lets equal bars share a rectangle correctly with the sentinels used here. Stay consistent with the width formula you chose.
- Forgetting the sentinel 0s and then special-casing an empty stack in the middle of the loop.
- Counting both sides of an equal minimum as strict. Equals get double-counted, and the modulo answer is wrong.

Practice

Stay before chapter 16. Then before you leave the chapter. Three problems are a second pass.

739. Daily Temperatures MEDIUM STAY
Stack of indices. The answer is the distance, and 0 means no warmer day.

496. Next Greater Element I EASY STAY
Next greater on nums2, then look up each value of nums1. A map from value to answer avoids a second search.

239. Sliding Window Maximum HARD STAY
Deque of indices, head pointer, pop the back while it is smaller or equal.

84. Largest Rectangle in Histogram HARD STAY
Sentinel 0s. Width is i − left − 1. The popped bar is the height.

503. Next Greater Element II MEDIUM THEN
Scan to 2n − 1. Push only while i < n.

901. Online Stock Span MEDIUM THEN
Stack of [price, span]. Pop prices that are <= today and add their spans.

1438. Longest Continuous Subarray With Absolute Diff Less Than or Equal to Limit MEDIUM THEN
One deque for the max, one for the min. Shrink while the difference exceeds limit.

862. Shortest Subarray with Sum at Least K HARD SECOND PASS
Prefix plus an increasing deque of prefix indices. Negatives are why the window died.

907. Sum of Subarray Minimums MEDIUM SECOND PASS
Strict previous smaller, non-strict next smaller. Contribution modulo 1e9+7.

456. 132 Pattern MEDIUM SECOND PASS
Right to left. ak is the best 2 popped by a larger 3. A later 1 that is smaller than ak finishes it.

REWRITE FROM MEMORY

Rewrite next-greater and the histogram with both sentinel zeros. Say why shift is not the deque operation.

Carry forward. Chapter 16 is a scan that keeps a reach or a tank, and never needs a stack.

Greedy scans

Keep the farthest reach. If the index you are standing on is past it, stop.

You can keep a stack in order. Some problems only need the farthest index you can currently reach. The invariant is a number, updated as you walk, and a failure is the moment the walk steps past that number.

RECOGNITION

- Jump game: from each index you may leap forward by at most nums[i].
- A gas circuit: if the total gas is less than the total cost, no start works. Otherwise one start does.
- Place a flower only when both neighbors are empty.
- Make an array non-decreasing with at most one change. The choice of which cell to edit depends on the cell two steps back.

SAY THIS BEFORE YOU CODE

The invariant: reach is the farthest index the jumps I have already seen can land on. If i itself is beyond reach, every later index is impossible too.

CHECK I AGAINST REACH BEFORE YOU EXTEND IT.

```
function canJump(nums) {
  let reach = 0;
  for (let i = 0; i < nums.length; i++) {
    if (i > reach) return false;
    reach = Math.max(reach, i + nums[i]);
  }
  return true;
}
```

```
[2, 3, 1, 1, 4]
i=0 reach = 2
i=1 reach = max(2, 1+3) = 4
i=2, 3, 4 are all <= 4
true

[3, 2, 1, 0, 4]
i=0 reach 3
i=1 reach 3
i=2 reach 3
i=3 reach 3
i=4 > 3, false
```

Jump II asks for the number of jumps. Think of the array as layers of a breadth-first search. `end` is the right edge of the layer you are currently jumping from. `far` is the furthest anything in this layer can reach. When `i` arrives at `end`, you must take one jump, and the next layer ends at `far`. Stop the loop at `n - 2`. The last index does not need a jump of its own.

ONE JUMP EACH TIME YOU EXHAUST THE CURRENT LAYER.

```
function jump(nums) {
  let jumps = 0;
  let end = 0;
  let far = 0;
  for (let i = 0; i < nums.length - 1; i++) {
    far = Math.max(far, i + nums[i]);
    if (i === end) {
      jumps += 1;
      end = far;
```

```
    }
  }
  return jumps;
}
```

The tank, the flower, the one edit

A NEGATIVE TANK MEANS EVERY INDEX UP TO I FAILS AS A START.

```
function canCompleteCircuit(gas, cost) {
  let total = 0;
  let tank = 0;
  let start = 0;
  for (let i = 0; i < gas.length; i++) {
    const gain = gas[i] - cost[i];
    total += gain;
    tank += gain;
    if (tank < 0) {
      start = i + 1;
      tank = 0;
    }
  }
  return total < 0 ? -1 : start;
}
```

If the total gas is at least the total cost, a circuit exists, and the start is the index after the last time the running tank went negative. Any earlier start dies at that same failure, because the tank arrived there even lower.

Can place flowers: a slot is free when it is 0 and both neighbors are 0 or off the end of the bed. Plant it, so the next check sees the new flower. Stop early when you have planted n. You may mutate the bed.

Non-decreasing array, at most one change. Walk from the left. On `nums[i] < nums[i - 1]`, you must spend your one change. If `i === 1` or `nums[i] >= nums[i - 2]`, lower `nums[i - 1]` down to `nums[i]`. That keeps the pair before i legal. Otherwise raise `nums[i]` up to `nums[i - 1]`, because lowering the previous cell would break the cell two steps back. A second violation returns false.

TRAPS

- Extending reach before you test `i > reach`. You would jump from an index you cannot stand on.
- Counting a jump at the last index in jump II. The loop ends at n − 2.
- Returning a start when total gas is below total cost. The local reset does not create fuel.
- Always lowering nums[i − 1] on a violation. [3, 4, 2, 3] needs you to notice that 2 is below nums[i − 2].

Practice

Stay before chapter 17. Then before you leave the chapter.

55. Jump Game MEDIUM STAY
If i > reach, return false. Then reach = max(reach, i + nums[i]).

45. Jump Game II MEDIUM STAY
Current end of the layer, and far. One jump when i hits end. Loop to n − 2.

134. Gas Station MEDIUM STAY
Total gas < total cost returns −1. Otherwise start after the last negative tank.

605. Can Place Flowers EASY THEN
Plant when the slot and both neighbors are empty. Decrement n as you plant.

665. Non-decreasing Array MEDIUM THEN

One change. Compare nums[i] with nums[i − 2] before you choose which cell to edit.

REWRITE FROM MEMORY

Rewrite jump I and the gas-station reset. Say which edit non-decreasing chooses, and why nums[i − 2] decides.

Carry forward. Chapter 17 does not add a template. It asks you to name, from the wording alone, which chapter you just finished.

Recognition sheet

A short list of questions ends in a chapter number. Say the number before you code.

You have a template for each of the last sixteen chapters. Recognition is the skill of picking one before you write it. Ask the questions in order. Stop at the first chapter that fits. Say that chapter number out loud, then say the brute cost, then code.

The questions

1. Is the input a grid? If the job is a region, a flood, or a distance in layers, use chapter 23. If a cell is the sum or the min of earlier cells, use chapter 24. If you sum a rectangle, use chapter 21. If each row and each column is sorted, or each row is sorted and the rows themselves are ordered, use chapter 22. If you walk, rotate, or spiral, use chapter 19. If the next state has to be stored in the cell, use chapter 20. Otherwise you are still indexing, chapter 18.
2. Does each index want the nearest greater or smaller neighbor, a span, a histogram rectangle, or the max of a window after the max can leave? Use a stack or a deque, chapter 15.
3. Are the values in 1..n or 0..n, and is the question a missing value, a duplicate, or a first positive? Place each value at its index, chapter 13. A single number among pairs is XOR, still chapter 13.
4. Is the thing you search a speed, a capacity, a day, or a boundary in a sorted array? Use the half-open loop, chapter 14.
5. Do you want the maximum sum of a contiguous segment, empty forbidden, with negatives allowed and no other constraint? Use Kadane, chapter 11.
6. Do you add a value onto many ranges and then read the array? Use a difference array, chapter 9.
7. Do you need a range sum, or the count of subarrays with sum K, and can a negative repair a sum? Use a prefix map, chapter 8.
8. Is the band contiguous, and does growing it only move you toward invalid while shrinking repairs it? Use a window, chapter 7.
9. Is the array sorted, or are you comparing two ends, so that one end can be discarded forever? Use pointers from the ends, chapter 5.
10. Are you overwriting the array, with a read index and a write index that both move right? Use chapter 6.
11. Do you need every pair in order, with duplicate triples removed, or are the items intervals? Sort, then scan, chapter 12.
12. Are you asking "have I seen the complement?" on unsorted data? Use a map, chapter 4.
13. Is n small enough that every pair or every subarray is acceptable? Use two loops, chapter 3.
14. Does the answer at i need only nums[i] and a summary of one side? Use one pass, chapter 2. A reach, a jump layer, or a tank is the greedy scan, chapter 16. Two full sides, left and right, are chapter 10.

Say the distinction in the affirmative

- Use a window when adding the next element only moves the range toward invalid, and dropping the left end repairs it. The values that the condition cares about are non-negative, or they are counts you can undo.
- Use a prefix map when a negative value can repair a sum you have already walked past, or when you need the number of subarrays whose sum is K.
- Use pointers from the ends when the array is sorted, or you are comparing two ends, and each step can discard one end forever.
- Use a window, rather than those ends, when both indices only move right and the answer is a contiguous band.

- Use Kadane when you want the best sum of a non-empty contiguous segment and a negative prefix should be thrown away. Use a window for a sum only when a separate bound exists and the values are non-negative.
- Use a hash map when the pair is unsorted and you need a complement in expected linear time.
- Use a sort when you need every pair in order, duplicates skipped, or the objects are intervals, and n log n is acceptable.
- Use binary search on an index when a sorted half tells you which side cannot hold the target.
- Use binary search on the answer when feasibility is monotone: a faster speed, a larger ship, a later day, and you can test one candidate in linear time.
- Use a stack when the answer at i is the nearest greater element, and a popped index can never be the answer for anyone later.
- Use a plain right-to-left pass when the answer is the greatest element anywhere to the right, not the nearest one.
- Use cyclic placement when each value in 1..n has a home index. Use a set when you only need membership and the values do not fit in the index range, or you must not write into the array.

Answer key for the eight drills

These are the prompts from chapter 1. Cover them on a reread only after you have written your own guess.

DRILL	USE	CHAPTER
Longest subarray, sum ≤ K, all positive	Variable window, shrink while the sum is too big	**07**
Count subarrays with sum K, negatives allowed	Prefix frequency, seed 0, query before update	**08**
Two indices, sorted, sum to target	Pointers from the ends	**05**
Remove duplicates from a sorted array, in place	Read r, write w	**06**
Next warmer day	Monotonic stack of indices, answer is the distance	**15**
Number of islands	Grid search, mark a cell when you enter it	**23**
Add a value on many ranges, then read	Difference array, +v at L and −v at R+1	**09**
Minimum eating speed	Binary search the speed, test hours in linear time	**14**

Twelve prompts

Cover the next page. For each prompt, write the pattern name and the chapter. These are original. They are not a problem statement for you to submit.

1. A string of 0s and 1s. You may flip at most K zeros. What is the longest all-ones stretch you can produce?
2. Values may be negative. How many contiguous segments sum to exactly K?
3. A sorted array. Find two indices that add to a target.
4. Daily prices. One buy and one later sell. Maximum profit.
5. Unsorted integers. The longest streak of consecutive values. The order in the array does not matter.
6. Split the array into at most M contiguous parts. Minimize the largest part sum.
7. For each day, how many days until a strictly warmer temperature?
8. Add v to every index in many inclusive ranges, then print the array.
9. A grid of land and water. How many connected groups of land are there, 4-directionally?
10. The contiguous segment with the largest sum. Negatives exist. The empty segment is forbidden.
11. Each row of a matrix is sorted, and the first cell of a row is greater than the last cell of the row above. Is target present?
12. A sorted array. Remove duplicates in place so each remaining value appears once.

Pattern names

Uncover this only after the twelve prompts have a written answer.

1. Sliding window, at most K zeros. Chapter 7.
2. Prefix frequency map. Chapter 8.
3. Pointers from the ends. Chapter 5.
4. One pass, minimum price so far, profit before the update. Chapter 2.
5. Set, start a run only when x − 1 is absent. Chapter 4.
6. Binary search on the answer. Chapter 14.
7. Monotonic stack. Chapter 15.
8. Difference array. Chapter 9.
9. Depth-first search, mark on entry. Chapter 23.
10. Kadane. Chapter 11.
11. Binary search on the flat index. Chapter 22.
12. Same-direction read and write. Chapter 6.

REWRITE FROM MEMORY

Close the page and walk the twelve prompts again. You are done when the chapter number comes out before the template does.

Carry forward. Chapter 18 starts the matrix. The index becomes a row and a column, and the shared-array bug from chapter 0 becomes visible.

CHAPTER 18 · BUILDS ON CHAPTER 0

The matrix from zero

matrix[r][c] is row then column. Allocate each row with its own array.

An array was one index. A matrix is an array of arrays. `matrix[r][c]` is the cell at row r, column c. There are `m = matrix.length` rows and `n = matrix[0].length` columns, once you know the matrix is non-empty. The last row is `m - 1`. The last column is `n - 1`.

Chapter 0 said an array is an object. This is the bug that sentence was saving:

ONE INNER ARRAY, SHARED, VERSUS A FRESH INNER ARRAY PER ROW.

```
const broken = Array(2).fill(Array(3).fill(0));
broken[0][0] = 9;
// broken[1][0] is 9 as well. fill copied one reference.

const rows = Array.from({ length: 2 }, () => Array(3).fill(0));
rows[0][0] = 9;
// rows[1][0] stays 0. Each call builds a new inner array.
```

SAY THIS BEFORE YOU CODE

The invariant: every row I allocate is a new array. I check r and c against m and n before I read.

Four loops, written cold

ROW-MAJOR, COLUMN-MAJOR, THE MAIN DIAGONAL, AND THE FOUR NEIGHBORS.

```
function eachRow(matrix) {
  const m = matrix.length;
  const n = matrix[0].length;
  for (let r = 0; r < m; r++) {
    for (let c = 0; c < n; c++) {
      const cell = matrix[r][c];
    }
  }
}

function eachColumn(matrix) {
  const m = matrix.length;
  const n = matrix[0].length;
  for (let c = 0; c < n; c++) {
    for (let r = 0; r < m; r++) {
      const cell = matrix[r][c];
    }
  }
}

function mainDiagonal(matrix) {
  const m = matrix.length;
  const n = matrix[0].length;
  for (let i = 0; i < Math.min(m, n); i++) {
    const cell = matrix[i][i];
  }
}

const DIRS = [[1, 0], [-1, 0], [0, 1], [0, -1]];

function inBounds(r, c, m, n) {
  return r >= 0 && c >= 0 && r < m && c < n;
}
```

The anti-diagonal of a square matrix is `matrix[i][n - 1 - i]`. A Toeplitz diagonal is constant when `matrix[r][c] === matrix[r - 1][c - 1]` for every cell that has a parent on that diagonal. Island

perimeter is a count, not a search: each land cell starts with 4 sides, and every shared edge with the land above or the land to the left removes 2, because both cells counted that edge.

Reshape is the flat index from chapter 14's matrix search, used earlier than the search. Cell number `i` lives at row `floor(i / n)` and column `i % n` in the old matrix, and at `floor(i / c)`, `i % c` in the new one. If `m * n` does not equal `r * c`, return the original matrix.

Count negatives in a sorted matrix can be a double loop today. Each row and each column is non-increasing. Chapter 22 walks it from a corner and does not revisit a cell. Brute force is the correct Stay if you have not practiced the staircase yet.

TRAPS

- `Array(m).fill(Array(n).fill(0))` aliases every row. The mutation shows up one row away from the write.
- Reading `matrix[0].length` on an empty matrix. Check `matrix.length` first.
- Swapping the meaning of r and c when you copy a column. A column is `matrix[r][c]` as r changes, not as c changes.
- Walking four directions without the bounds test. The neighbor of a border cell is not a cell.

Practice

Stay before chapter 19. Then before you leave the chapter.

1672. Richest Customer Wealth EASY STAY
Each row is a customer. Sum the row, keep the max.

1572. Matrix Diagonal Sum EASY STAY
Add both diagonals. On an odd n, the center was added twice. Subtract it once.

766. Toeplitz Matrix EASY STAY
Every cell with a parent must equal matrix[r - 1][c - 1].

566. Reshape the Matrix EASY STAY
Refuse when the counts differ. Map flat i through the old width and the new width.

1380. Lucky Numbers in a Matrix EASY THEN
A lucky number is the min of its row and the max of its column. Brute force is appropriate.

118. Pascal's Triangle EASY THEN
Row 0 is [1]. Each later cell is the sum of the two parents, and a missing parent is 0.

463. Island Perimeter EASY THEN
4 per land, minus 2 for each land neighbor above or to the left.

1351. Count Negative Numbers in a Sorted Matrix EASY THEN
A double loop is fine here. Chapter 22 revisits it as a staircase from the top-right.

REWRITE FROM MEMORY

Rewrite the four loops and the Array.from allocation from a blank file. Then mutate one shared-row matrix on paper.

Carry forward. Chapter 19 walks those cells in an order: a diagonal, a spiral, a transpose, a rotation.

CHAPTER 19 · BUILDS ON CHAPTER 18

Walk the matrix

A spiral keeps four edges, and it refuses a leg that would walk a row or a column twice.

You can visit every cell. The interview question is the order, and the bug is visiting a cell twice when the matrix is a single row or a single column.

RECOGNITION

- Diagonal order alternates direction, and a corner has to prefer one wall over the other.
- Spiral peels a layer: right, down, left, up, and each leg checks that the layer is still alive.
- Rotate 90 degrees clockwise is a transpose plus a reversal of each row.
- A score that flips rows and columns is a greedy choice per column, after you force the first column to be 1s.

SAY THIS BEFORE YOU CODE

The invariant: top, bottom, left, and right are the edges of the layer I have not walked. I only walk a leg when that layer still has a row, or still has a column.

Transpose, flip, rotate

TRANSPOSE ACROSS THE MAIN DIAGONAL, THEN REVERSE EACH ROW.

```
function rotate(matrix) {
  const n = matrix.length;
  for (let r = 0; r < n; r++) {
    for (let c = r + 1; c < n; c++) {
      const tmp = matrix[r][c];
      matrix[r][c] = matrix[c][r];
      matrix[c][r] = tmp;
    }
  }
  for (const row of matrix) row.reverse();
}
```

```
1 2 3        1 4 7        7 4 1
4 5 6   →    2 5 8   →    8 5 2
7 8 9        3 6 9        9 6 3
transpose swaps (r, c) with (c, r) only when c > r
reverse each row
```

Flipping an image reverses each row and inverts each bit. Do both in one swap from the ends: the new left value is the old right value XOR 1, and the new right value is the old left value XOR 1. When the two indices meet, XOR that single cell.

Diagonal, then spiral

Diagonal traverse alternates an up-right step and a down-left step. The order of the wall checks matters. Going up, test the right edge before the top edge, so the corner cell steps down instead of trying to leave through the top. Going down, test the bottom edge before the left edge.

THE LAST TWO LEGS ARE GUARDED, SO A SINGLE ROW IS NOT WALKED BACKWARD.

```
function spiralOrder(matrix) {
  const out = [];
  let top = 0;
  let bot = matrix.length - 1;
  let left = 0;
  let right = matrix[0].length - 1;
  while (top <= bot && left <= right) {
    for (let c = left; c <= right; c++) out.push(matrix[top][c]);
```

```
    top += 1;
    for (let r = top; r <= bot; r++) out.push(matrix[r][right]);
    right -= 1;
    if (top <= bot) {
      for (let c = right; c >= left; c--) out.push(matrix[bot][c]);
      bot -= 1;
    }
    if (left <= right) {
      for (let r = bot; r >= top; r--) out.push(matrix[r][left]);
      left += 1;
    }
  }
  return out;
}
```

```
1 2 3
4 5 6
7 8 9
right along top → 1 2 3, top becomes 1
down the right → 6 9, right becomes 1
left along bot, top 1 <= bot 2 → 8 7, bot becomes 1
up the left, left 0 <= right 1 → 4, left becomes 1
right along the middle row → 5
[1, 2, 3, 6, 9, 8, 7, 4, 5]

a single row [1, 2, 3]: the rightward leg takes all three,
top moves past bot, and the leftward leg is skipped
```

Spiral II writes the same four legs into a fresh square, placing the numbers 1 through n^2. Allocate it with `Array.from`, not with a shared row.

Score after flipping

The first column is worth the most. Flip any row that starts with 0, so column 0 becomes all ones. That row flip also flips every later bit in the row. A later column can then be flipped as a whole. For column c, count how many cells would be 1 after the row flips you already committed to: a cell matches `grid[r][0]` exactly when the row flip leaves it as 1. Take the max of that count and its complement, and add it times the place value `1 << (n - 1 - c)`. The answer's leading term is `m` times the highest place, because column 0 is all ones.

TRAPS

- Walking the spiral's leftward leg after the top row was the only row. You emit the row twice, reversed.
- On the diagonal walk, testing the top edge before the right edge. The top-right corner steps the wrong way and you loop or you skip.
- Rotating by four-cycles without a worked example. Transpose then reverse is the version you can rewrite. The cycle version is optional once this one is boring.
- Building the spiral matrix with a shared inner array. The writes alias.

Practice

Stay before chapter 20. Then before you leave the chapter.

54. Spiral Matrix MEDIUM STAY
Four legs. Guard the leftward and upward legs with the updated edges.

48. Rotate Image MEDIUM STAY
Transpose, then reverse each row. Square matrix, in place.

498. Diagonal Traverse MEDIUM STAY
Up-right and down-left. Hit the far edge of the current direction before the near edge.

867. Transpose Matrix EASY THEN

out[c][r] = matrix[r][c]. The result has n rows and m columns. Allocate it fresh.

832. Flipping an Image EASY THEN

Reverse and invert in one end-to-end swap. XOR 1 flips a bit.

59. Spiral Matrix II MEDIUM THEN

The same four legs, writing the next integer. One row and one column guards still apply.

861. Score After Flipping Matrix MEDIUM THEN

Force column 0 to ones by row flips. Then each later column takes the majority bit.

REWRITE FROM MEMORY

Rewrite the spiral, including both guards, and the rotate: transpose, then reverse each row.

Carry forward. Chapter 20 writes a mark into the cell itself, because you are not allowed a second matrix.

In-place marks

Store the second fact in the cell, or in the first row, and do not erase a mark that a later cell still needs.

You can allocate a second matrix. Some problems forbid it, or they expect you not to. The cell has to remember two facts at once: what it was, and what it will become. A bit, a sign, or the first row used as a bulletin board, holds the second fact.

RECOGNITION

- Set zeroes, constant extra memory. The first row and the first column are the marks.
- Game of life, in place. The next state fits in bit 1. Neighbors must still see bit 0.
- Battleships that do not touch. Count only the top-left cell of each ship.
- Sudoku validity is three sets per row, per column, and per box. The solver, later, reuses those sets.

SAY THIS BEFORE YOU CODE

The invariant: I will not destroy a mark until every cell that needs to read it has already been written.

Zeroes, written from the bottom-right

COLUMN 0 HAS ITS OWN FLAG. THE WRITE-BACK STARTS AT THE BOTTOM-RIGHT.

```
function setZeroes(matrix) {
  const m = matrix.length;
  const n = matrix[0].length;
  let col0 = false;
  for (let r = 0; r < m; r++) {
    if (matrix[r][0] === 0) col0 = true;
    for (let c = 1; c < n; c++) {
      if (matrix[r][c] === 0) {
        matrix[r][0] = 0;
        matrix[0][c] = 0;
      }
    }
  }
  for (let r = m - 1; r >= 0; r--) {
    for (let c = n - 1; c >= 1; c--) {
      if (matrix[r][0] === 0 || matrix[0][c] === 0) matrix[r][c] = 0;
    }
    if (col0) matrix[r][0] = 0;
  }
}
```

The first pass stores "this row must die" in column 0 and "this column must die" in row 0. Column 0 cannot do both jobs, so a boolean remembers whether column 0 itself contained a zero. The write-back walks from the bottom. Rows below row 0 read `matrix[0][c]` while those marks are still intact. Row 0 is zeroed last. Inside a row, columns run from the right down to column 1, and column 0 is applied from the flag after the row mark has been read.

Life in bit 1

NEIGHBORS READ BIT 0. BIT 1 IS THE NEXT STATE. SHIFT ONLY AFTER THE WHOLE BOARD IS DECIDED.

```
function gameOfLife(board) {
  const m = board.length;
  const n = board[0].length;
  const dirs = [
    [-1, -1], [-1, 0], [-1, 1],
    [0, -1], [0, 1],
    [1, -1], [1, 0], [1, 1],
```

```
  ];
  for (let r = 0; r < m; r++) {
    for (let c = 0; c < n; c++) {
      let live = 0;
      for (const [dr, dc] of dirs) {
        const nr = r + dr;
        const nc = c + dc;
        if (nr < 0 || nc < 0 || nr >= m || nc >= n) continue;
        if ((board[nr][nc] & 1) === 1) live += 1;
      }
      const cur = board[r][c] & 1;
      if ((cur === 1 && (live === 2 || live === 3)) || (cur === 0 && live === 3)) {
        board[r][c] |= 2;
      }
    }
  }
  for (let r = 0; r < m; r++) {
    for (let c = 0; c < n; c++) board[r][c] >>= 1;
  }
}
```

A live cell with two or three live neighbors stays live. A dead cell with three becomes live. Everyone else is dead, which is a clear bit 1, so you only set bit 1 in the two surviving cases. The final shift drops the old state.

Ships and sudoku

Battleships are horizontal or vertical and they do not touch. The top-left cell of a ship is an `X` whose upper neighbor is not `X` and whose left neighbor is not `X`. Counting those cells counts the ships. You never flood.

Valid sudoku: nine sets for rows, nine for columns, nine for boxes. The box index is `floor(r / 3) * 3 + floor(c / 3)`. Skip dots. A digit that is already in its row, its column, or its box makes the board invalid. You are checking a filled board, not solving it. The solver is chapter 23, and it uses these same three families of sets.

TRAPS

- Zeroing row 0 before the other rows have read it. The column marks disappear.
- Using column 0 both as a row-mark and as the "column 0 is zero" fact, with no separate flag.
- Letting neighbors in Game of Life read the whole cell after you have stored the next state in the same number, without masking bit 0.
- Shifting each row as soon as you finish it. The row below would see the new state.
- Counting every `X` as a battleship. Only the top-left cell of each ship.

Practice

Stay before chapter 21. Then before you leave the chapter.

73. Set Matrix Zeroes MEDIUM STAY
First row and column are marks, plus a flag for column 0. Write back from the bottom-right.

289. Game of Life MEDIUM STAY
Next state in bit 1. Neighbors use board[r][c] & 1. Shift after the full pass.

36. Valid Sudoku MEDIUM STAY
A set per row, per column, and per 3×3 box. Dots are empty.

419. Battleships in a Board MEDIUM THEN
Count an X only when the cell above and the cell to the left are not X.

REWRITE FROM MEMORY

Rewrite set-zeroes from the bottom-right, and the life update that masks with & 1 before the shift.

Carry forward. Chapter 21 goes back to prefix sums. The prefix becomes a grid, and a rectangle is four corners.

CHAPTER 21 · BUILDS ON CHAPTERS 8 AND 18

2D prefix

A rectangle is four corners of the prefix. Add back the top-left, because you subtracted it twice.

You can sum a 1D range with `prefix[j + 1] − prefix[i]`. A matrix prefix does the same for a rectangle. Let `p[i + 1][j + 1]` be the sum of the rectangle from `(0, 0)` through `(i, j)`, inclusive. It is built from the cell plus the rectangle above plus the rectangle to the left, minus the rectangle above-left, which was included twice.

RECOGNITION

- Many rectangle-sum queries on a matrix that does not change.
- Each cell answers the sum of a square around itself. Build the prefix once.
- Count submatrices that sum to a target. Fix two horizontal edges, collapse the rows between them into a 1D array, and reuse the chapter 8 frequency map.

SAY THIS BEFORE YOU CODE

The invariant: p[r][c] is the sum of every cell strictly above row r and strictly left of column c, in the original matrix. I add back the top-left corner because I subtracted it twice.

INCLUSIVE RECTANGLE (R1, C1) THROUGH (R2, C2). THE LAST TERM IS THE ONE YOU SAY OUT LOUD.

```
function buildPrefix(mat) {
  const m = mat.length;
  const n = mat[0].length;
  const p = Array.from({ length: m + 1 }, () => Array(n + 1).fill(0));
  for (let i = 0; i < m; i++) {
    for (let j = 0; j < n; j++) {
      p[i + 1][j + 1] = mat[i][j] + p[i][j + 1] + p[i + 1][j] − p[i][j];
    }
  }
  return p;
}

function rectSum(p, r1, c1, r2, c2) {
  return p[r2 + 1][c2 + 1] − p[r1][c2 + 1] − p[r2 + 1][c1] + p[r1][c1];
}
```

```
mat
1 2
3 4

p, with a zero border
0  0  0
0  1  3
0  4  10

whole matrix: p[2][2] − p[0][2] − p[2][0] + p[0][0] = 10
bottom-right cell only, (1,1)-(1,1):
10 − p[1][2] − p[2][1] + p[1][1] = 10 − 3 − 4 + 1 = 4
the +1 is the top-left rectangle that both subtractions removed
```

Build is O(mn). Each rectangle query is O(1). Block sum over every cell is O(mn). Counting submatrices that sum to a target, by fixing two rows, is O(m² n).

Matrix block sum asks each cell for the sum of the square that extends k cells out, clipped to the matrix. Clamp the rectangle, then call `rectSum`. Do not rebuild a prefix per cell.

This is the point of the path

Number of submatrices that sum to target: choose the top row and the bottom row. Add those rows into a 1D array `col`, one entry per column. A contiguous segment of `col` is exactly a submatrix between those two rows. Count the segments whose sum is the target with the chapter 8 map: seed 0 → 1, query `sum - target`, then increment. Move the bottom row down by adding the next matrix row into `col`, and rebuild the map. The 1D algorithm did not change. The matrix only chose which 1D array you hand it.

SECOND PASS, AND THE TEMPLATE IS COMPLETE. TWO ROWS, THEN CHAPTER 8.

```
function numSubmatrixSumTarget(matrix, target) {
  const m = matrix.length;
  const n = matrix[0].length;
  let count = 0;
  for (let top = 0; top < m; top++) {
    const col = new Array(n).fill(0);
    for (let bot = top; bot < m; bot++) {
      for (let c = 0; c < n; c++) col[c] += matrix[bot][c];
      const freq = new Map();
      freq.set(0, 1);
      let sum = 0;
      for (const x of col) {
        sum += x;
        count += freq.get(sum - target) ?? 0;
        freq.set(sum, (freq.get(sum) ?? 0) + 1);
      }
    }
  }
  return count;
}
```

TRAPS

- Forgetting the added-back term. The two subtractions each removed the top-left rectangle.
- Allocating the prefix with a shared inner array. Every row aliasing makes the build nonsense.
- Using `p[r2][c2]` for an inclusive end. The prefix is one larger than the matrix in each dimension. The inclusive end lives at `r2 + 1`, `c2 + 1`.
- Rebuilding the 1D frequency map and forgetting the seed, now that the map is nested inside two row loops.

Practice

Stay before chapter 22. The collapse to 1D is a second pass, and the template above is the one to adapt.

304. Range Sum Query 2D - Immutable MEDIUM STAY
Build p in the constructor. sumRegion is the four-term formula.

1314. Matrix Block Sum MEDIUM STAY
One prefix. Each cell clamps [i − k, j − k] through [i + k, j + k] to the matrix.

1074. Number of Submatrices That Sum to Target HARD SECOND PASS
Fix two rows, collapse to a 1D array, run the chapter 8 count.

REWRITE FROM MEMORY

Rewrite the build and rectSum until you can say the added-back term without looking.

Carry forward. Chapter 22 searches a sorted matrix. One version treats it as a flat sorted array. The other starts at a corner that has a direction.

CHAPTER 22 · BUILDS ON CHAPTERS 14 AND 18

Search a sorted matrix

A fully ordered matrix is a flat binary search. A row-and-column sorted matrix starts at the top-right.

You can sum a rectangle. Searching a sorted matrix is a different property. There are two properties, and they are not the same problem.

RECOGNITION

- Each row is sorted left to right, and the first cell of each row is greater than the last cell of the previous row. The matrix is one sorted array written in rows. Binary-search the flat index.
- Each row is sorted, and each column is sorted, but the row-to-row property is absent. A flat index is not sorted. Start at a corner where one step decreases and the other step increases.
- You want the kth smallest value in a row-and-column sorted matrix. Binary-search the value, and count how many entries are <= mid.

SAY THIS BEFORE YOU CODE

The invariant for the staircase: every cell above the current row is too small, and every cell to the right of the current column is too large. The target, if it exists, is still in the rectangle I have not discarded.

The flat index

CHAPTER 14'S LOWER BOUND, ON AN INDEX THAT RUNS FROM 0 TO M*N.

```
function searchMatrix(matrix, target) {
  const m = matrix.length;
  const n = matrix[0].length;
  let lo = 0;
  let hi = m * n;
  while (lo < hi) {
    const mid = lo + Math.floor((hi - lo) / 2);
    const val = matrix[Math.floor(mid / n)][mid % n];
    if (val >= target) hi = mid;
    else lo = mid + 1;
  }
  if (lo === m * n) return false;
  return matrix[Math.floor(lo / n)][lo % n] === target;
}
```

```
1  3  5
7  9  11
target 9, n = 3, flat length 6
[0, 6) mid 3 → row 1 col 0 → 7, 7 < 9, lo = 4
[4, 6) mid 5 → row 1 col 2 → 11, 11 >= 9, hi = 5
[4, 5) mid 4 → row 1 col 1 → 9, hi = 4
lo === 4, matrix[1][1] === 9, true
```

The corner that has a direction

Start at the top-right. In a matrix with sorted rows and sorted columns, moving left decreases, and moving down increases. If the cell is too big, move left. If it is too small, move down. Equal means you found it. The top-left corner does not give you a direction: both right and down are larger, so a miss does not tell you which way the target went.

TOP-RIGHT. TOO BIG GOES LEFT. TOO SMALL GOES DOWN.

```
function searchMatrix(matrix, target) {
  const m = matrix.length;
  const n = matrix[0].length;
  let r = 0;
```

```
  let c = n - 1;
  while (r < m && c >= 0) {
    if (matrix[r][c] === target) return true;
    if (matrix[r][c] > target) c -= 1;
    else r += 1;
  }
  return false;
}
```

```
1  4  7
2  5  8
3  6  9
target 6, start at 7
7 > 6, left to 4
4 < 6, down to 5
5 < 6, down to 6
found

start at 1 instead: 1 < 6, and both the right neighbor and the lower neighbor are larger
you would be guessing
```

The flat search is O(log (mn)). The staircase is O(m + n). The kth-smallest search is O(n log V) on an n by n matrix, where V is the range of values.

Count negatives, the problem you brute-forced in chapter 18, is this staircase on a matrix that is sorted non-increasing in each row and each column. Start at the top-right. If the cell is negative, every cell below it in that column is negative too, so add `m - r` and move left. If it is not negative, move down.

Kth smallest: the predicate "how many entries are <= mid" is monotone in mid. Low is `matrix[0][0]`. High is `matrix[n - 1][n - 1]`, and this loop treats the value range as integers, stopping when low equals high. Count by walking each row from the right, carrying the column cursor down so the count stays linear per test. `ok(mid)` means the count is at least k, which sets `hi = mid`.

SEARCH THE VALUE. THE SAME LO < HI PROGRESS AS CHAPTER 14.

```
function kthSmallest(matrix, k) {
  const n = matrix.length;
  let lo = matrix[0][0];
  let hi = matrix[n - 1][n - 1];
  const countAtMost = (mid) => {
    let count = 0;
    let c = n - 1;
    for (let r = 0; r < n; r++) {
      while (c >= 0 && matrix[r][c] > mid) c -= 1;
      count += c + 1;
    }
    return count;
  };
  while (lo < hi) {
    const mid = lo + Math.floor((hi - lo) / 2);
    if (countAtMost(mid) >= k) hi = mid;
    else lo = mid + 1;
  }
  return lo;
}
```

TRAPS

- Using the flat binary search on a matrix that is only row-sorted and column-sorted. The flat sequence is not sorted.
- Starting the staircase at the top-left. Both legal steps increase.
- Starting at the bottom-left and swapping the comparisons without re-deriving them. Bottom-left also works if you restate it: too small goes right, too big goes up. Memorize one corner.
- Forgetting that 1351 is non-increasing, so "negative" adds the cells below and moves left. The 240 comparisons are for non-decreasing rows and columns.

Practice

Stay before chapter 23. Then before you leave the chapter. Revisit 1351 with the staircase once 240 is boring.

74. Search a 2D Matrix MEDIUM STAY
Rows are sorted and stacked in order. Flat index mid / n and mid % n.

240. Search a 2D Matrix II MEDIUM STAY
Rows and columns each sorted. Start at the top-right.

378. Kth Smallest Element in a Sorted Matrix MEDIUM THEN
Binary-search the value. Count entries <= mid, one descending cursor per test.

REWRITE FROM MEMORY

Rewrite both searches. Say why the top-left corner does not tell you which way to step.

Carry forward. Chapter 23 treats the matrix as a graph. A cell has up to four neighbors, and you mark a cell when you enter it.

CHAPTER 23 · BUILDS ON CHAPTERS 18 AND 20

Regions and grid graphs

Mark a cell when you enter it. A layer of the queue is one step.

A neighbor is an edge. The matrix is a graph, drawn as a grid so you do not build an adjacency list. The bug that doubles work is marking a cell when you leave it. Mark it when you enter, before you walk its neighbors. A breadth-first search marks when it enqueues, for the same reason: a cell waiting in the queue is already visited.

RECOGNITION

- How many components, how large is this component, recolor this region. Depth-first search, mark on entry.
- Minutes to rot, distance to the nearest 0, the shortest bridge. Breadth-first search from every source at once. A layer is one minute, or one step.
- A cell that can reach both oceans. Start from the ocean borders and walk uphill, to a neighbor greater than or equal.
- A word on the board. Mark, recurse, unmark, because the same cell cannot be reused in one word and must be reusable in the next attempt.

SAY THIS BEFORE YOU CODE

The invariant: a cell I have entered will not be entered again. For a search that backtracks, the mark lasts only for this path, and the unmark restores it.

Islands

WRITE THE MARK BEFORE THE RECURSIVE CALLS.

```
function numIslands(grid) {
  const m = grid.length;
  const n = grid[0].length;
  const dirs = [[1, 0], [-1, 0], [0, 1], [0, -1]];
  const dfs = (r, c) => {
    if (r < 0 || c < 0 || r >= m || c >= n || grid[r][c] !== "1") return;
    grid[r][c] = "0";
    for (const [dr, dc] of dirs) dfs(r + dr, c + dc);
  };
  let count = 0;
  for (let r = 0; r < m; r++) {
    for (let c = 0; c < n; c++) {
      if (grid[r][c] === "1") {
        count += 1;
        dfs(r, c);
      }
    }
  }
  return count;
}
```

Max area returns the size from the same function: 1 plus the sizes of the four calls, and a rejected neighbor returns 0. Flood fill is the same walk, painting `color`, and it returns immediately when the start cell is already that color. Otherwise the walk never terminates, because the new color equals the color you are following.

Closed islands: land is 0, water is 1. First sink every land cell that touches the border, by the same flood. Then count the remaining land components. Those are the islands that do not touch the border.

Surrounded regions: flood from every border `O`, marking those cells as a temporary letter. They are connected to the outside. Then walk the board. An untouched `O` becomes `X`. The temporary letter becomes `O` again.

A layer is one minute

EVERY ROTTEN ORANGE STARTS IN THE QUEUE. MARK A FRESH ORANGE WHEN YOU ENQUEUE IT.

```
function orangesRotting(grid) {
  const m = grid.length;
  const n = grid[0].length;
  const q = [];
  let fresh = 0;
  for (let r = 0; r < m; r++) {
    for (let c = 0; c < n; c++) {
      if (grid[r][c] === 2) q.push([r, c]);
      else if (grid[r][c] === 1) fresh += 1;
    }
  }
  const dirs = [[1, 0], [-1, 0], [0, 1], [0, -1]];
  let head = 0;
  let minutes = 0;
  while (head < q.length && fresh > 0) {
    const layer = q.length - head;
    for (let k = 0; k < layer; k++) {
      const [r, c] = q[head];
      head += 1;
      for (const [dr, dc] of dirs) {
        const nr = r + dr;
        const nc = c + dc;
        if (nr < 0 || nc < 0 || nr >= m || nc >= n || grid[nr][nc] !== 1) continue;
        grid[nr][nc] = 2;
        fresh -= 1;
        q.push([nr, nc]);
      }
    }
    minutes += 1;
  }
  return fresh === 0 ? minutes : -1;
}
```

```
minute 0, queue holds every 2
each loop processes exactly the current layer, then minutes += 1
an orange set to 2 before the push cannot be enqueued by a second neighbor
no fresh at the start → the while does not run → 0
fresh left and the queue is empty → -1
```

01 Matrix is the same multi-source search. Every 0 starts in the queue at distance 0. The first time you reach a cell, the distance is final, because breadth-first search walks in order of distance. A head pointer replaces `shift`.

Pacific Atlantic: a cell can flow to an ocean when there is a path of non-increasing heights to that ocean. Reverse the edges. From each ocean border, walk to a neighbor whose height is greater than or equal. A cell reached from both floods is an answer. Mark on entry so a cell is not expanded twice per ocean.

The bridge returns 0 when the islands already touch

PAINT ONE ISLAND. THE FIRST TIME THE EXPANSION TOUCHES A 1, RETURN THE STEPS SO FAR.

```
function shortestBridge(grid) {
  const n = grid.length;
  const dirs = [[1, 0], [-1, 0], [0, 1], [0, -1]];
  const q = [];
  const paint = (r, c) => {
    if (r < 0 || c < 0 || r >= n || c >= n || grid[r][c] !== 1) return;
    grid[r][c] = 2;
    q.push([r, c]);
    for (const [dr, dc] of dirs) paint(r + dr, c + dc);
  };
  let found = false;
  for (let r = 0; r < n && !found; r++) {
```

```
    for (let c = 0; c < n && !found; c++) {
      if (grid[r][c] === 1) {
        paint(r, c);
        found = true;
      }
    }
  }
  let head = 0;
  let steps = 0;
  while (head < q.length) {
    const layer = q.length - head;
    for (let k = 0; k < layer; k++) {
      const [r, c] = q[head];
      head += 1;
      for (const [dr, dc] of dirs) {
        const nr = r + dr;
        const nc = c + dc;
        if (nr < 0 || nc < 0 || nr >= n || nc >= n || grid[nr][nc] === 2) continue;
        if (grid[nr][nc] === 1) return steps;
        grid[nr][nc] = 2;
        q.push([nr, nc]);
      }
    }
    steps += 1;
  }
  return steps;
}
```

If the other island shares an edge with the painted one, that touch happens in the first layer, while `steps` is still 0. You flip zero water cells. A single water cell between them returns 1, because you enqueue the water, increment steps, and then see the other island.

Mark, recurse, unmark

THE MARK IS ONLY FOR THIS PATH. PUT THE LETTER BACK ON THE WAY OUT.

```
function exist(board, word) {
  const m = board.length;
  const n = board[0].length;
  const dfs = (r, c, i) => {
    if (i === word.length) return true;
    if (r < 0 || c < 0 || r >= m || c >= n || board[r][c] !== word[i]) return false;
    const saved = board[r][c];
    board[r][c] = "#";
    const ok = dfs(r + 1, c, i + 1) || dfs(r - 1, c, i + 1)
      || dfs(r, c + 1, i + 1) || dfs(r, c - 1, i + 1);
    board[r][c] = saved;
    return ok;
  };
  for (let r = 0; r < m; r++) {
    for (let c = 0; c < n; c++) {
      if (dfs(r, c, 0)) return true;
    }
  }
  return false;
}
```

Sudoku solver, second pass, is the same shape on digits. Collect the empty cells. Keep the three families of sets from chapter 20. Try a digit, add it to the row, the column, and the box, and recurse to the next empty cell. On failure, delete it from the three sets and clear the cell. The first time you pass the last empty cell, the board is solved.

Word search II is the same board walk with a dictionary. The dictionary is a trie of the words, so a prefix that no word uses can stop the recursion. Build the trie, walk the board, mark and unmark as above, and record a word

when a node says a word ends. That problem is a later lesson. The link is enough until you have a day for tries: leetcode.com/problems/word-search-ii/.

TRAPS

- Marking a cell after the recursive calls. The neighbor walks back in, and the component is counted many times, or the search never ends.
- Enqueuing a rotting orange without setting it to 2. The other neighbor enqueues it again, and the minute count lies.
- Using `Array.shift` for the queue. A head index is O(1).
- Forgetting to unmark in word search. A later path sees a hole that is really a letter.
- Flood fill into the same color. Guard it.
- Returning 1 from the bridge when the islands already share an edge. The first layer sees the other island at steps 0.

Practice

Stay before chapter 24. Then before you leave the chapter. The solver and the trie are a second pass.

200. Number of Islands MEDIUM STAY
Count starts of a search. Mark the land as water when you enter.

733. Flood Fill EASY STAY
If the new color equals the start color, return. Otherwise paint as you enter.

994. Rotting Oranges MEDIUM STAY
Multi-source. A layer is a minute. Return −1 if fresh oranges remain.

79. Word Search MEDIUM STAY
Mark, recurse, unmark. The letter must match word[i] before you mark.

695. Max Area of Island MEDIUM THEN
The island function returns 1 plus the four neighbors, or 0 when rejected.

130. Surrounded Regions MEDIUM THEN
Flood border O cells first. Flip only the O cells you never reached.

542. 01 Matrix MEDIUM THEN
Every 0 is a source at distance 0. The first visit is the distance.

417. Pacific Atlantic Water Flow MEDIUM THEN
Walk uphill from each ocean. A cell reached by both searches is an answer.

934. Shortest Bridge MEDIUM THEN
Paint one island, expand, return 0 if the other island already touches.

1254. Number of Closed Islands MEDIUM THEN
0 is land. Sink the border lands, then count what remains.

37. Sudoku Solver HARD SECOND PASS
Empty cells, three sets, place a digit, recurse, remove it.

212. Word Search II HARD SECOND PASS
The board walk you know, pruned by a trie. Learn the trie later. The link is the note above.

REWRITE FROM MEMORY

Rewrite islands and rotting oranges. Include the mark-before-enqueue, and the bridge's return of 0.

Carry forward. Chapter 24 fills a cell only after the cells it depends on. That filling order is the first dynamic programming in this book.

CHAPTER 24 · BUILDS ON CHAPTERS 15 AND 23

Cells that depend on smaller cells

Fill a cell after the cells it reads. The order is the algorithm.

A search walked cells in whatever order the stack or the queue produced. Some answers are simpler than a search. The cell's answer is a function of two or three neighbors that are already finished, if you fill the grid in the right order. This is dynamic programming. You do not need a course to use it. You need the order.

RECOGNITION

- Unique paths: a cell is the sum of the cell above and the cell to the left. The first row and the first column are 1, because there is only one way to hug the edge.
- An obstacle zeros a cell, and it also blocks the edge prefix that would have flowed through it.
- A path sum takes the min of the incoming neighbors, then adds the cell.
- A square of 1s grows by one plus the minimum of the three neighbors. The answer is the side times the side, and the matrix stores characters.
- A rectangle of 1s is a histogram per row. The histogram function is chapter 15.

SAY THIS BEFORE YOU CODE

The invariant: when I write cell (r, c), every cell it reads has already been written. I choose an order that makes that true: row by row, left to right, or a memoized search that only steps to a strictly larger neighbor.

Paths

EDGES STAY 1. EVERYONE ELSE ADDS THE TOP AND THE LEFT.

```
function uniquePaths(m, n) {
  const dp = Array.from({ length: m }, () => Array(n).fill(1));
  for (let r = 1; r < m; r++) {
    for (let c = 1; c < n; c++) {
      dp[r][c] = dp[r - 1][c] + dp[r][c - 1];
    }
  }
  return dp[m - 1][n - 1];
}
```

```
m = 3, n = 3, start filled with 1s
r=1,c=1: 1+1 = 2
r=1,c=2: 1+2 = 3
r=2,c=1: 2+1 = 3
r=2,c=2: 3+3 = 6
six ways across a 3 by 3 grid of cells
```

Unique paths II: an obstacle at the start returns 0. The first row is a prefix of 1s that becomes 0 at the first obstacle and stays 0. The first column works the same way. Every other obstacle writes 0. Every open cell adds the top and the left, which may themselves be 0.

THE FIRST ROW AND THE FIRST COLUMN ARE PREFIXES. EVERY OTHER CELL ADDS THE CHEAPER PARENT.

```
function minPathSum(grid) {
  const m = grid.length;
  const n = grid[0].length;
  const dp = Array.from({ length: m }, () => Array(n).fill(0));
  dp[0][0] = grid[0][0];
  for (let c = 1; c < n; c++) dp[0][c] = dp[0][c - 1] + grid[0][c];
  for (let r = 1; r < m; r++) {
    dp[r][0] = dp[r - 1][0] + grid[r][0];
    for (let c = 1; c < n; c++) {
      dp[r][c] = grid[r][c] + Math.min(dp[r - 1][c], dp[r][c - 1]);
```

```
    }
  }
  return dp[m - 1][n - 1];
}
```

Minimum falling path allows a step to the cell directly above or either diagonal above. Edges have only two parents, or one. Copy the matrix, then fill from row 1 downward, reading only the previous row. The answer is the minimum of the last row.

Squares, rectangles, and a rising path

THE CELLS ARE CHARACTERS. THE ANSWER IS THE AREA, SIDE TIMES SIDE.

```
function maximalSquare(matrix) {
  const m = matrix.length;
  const n = matrix[0].length;
  const dp = Array.from({ length: m }, () => Array(n).fill(0));
  let side = 0;
  for (let r = 0; r < m; r++) {
    for (let c = 0; c < n; c++) {
      if (matrix[r][c] !== "1") continue;
      const up = r > 0 ? dp[r - 1][c] : 0;
      const left = c > 0 ? dp[r][c - 1] : 0;
      const diag = r > 0 && c > 0 ? dp[r - 1][c - 1] : 0;
      dp[r][c] = 1 + Math.min(up, left, diag);
      if (dp[r][c] > side) side = dp[r][c];
    }
  }
  return side * side;
}
```

```
1 0 1
1 1 1
1 1 1
side at (0,0) = 1
(1,1) = 1 + min(1, 1, 0) = 1
(1,2) = 1 + min(0, 1, 1) = 1
(2,1) = 1 + min(1, 1, 1) = 2
(2,2) = 1 + min(1, 2, 1) = 2
area = 4
```

Maximal rectangle, second pass: each row builds a histogram. If `row[c]` is the character `"1"`, the height at column c grows by 1. A `"0"` resets that height to 0. The largest rectangle inside this histogram is the chapter 15 function, sentinels included. The answer is the best histogram across all rows. Paste that function. Do not re-derive the width.

Longest increasing path, second pass: depth-first search with a memo. From a cell, step to a strictly greater neighbor. The length at a cell is 1 plus the best neighbor, or 1 if it has none. Strict increase means the walk cannot cycle, so the memo is only a cache, not a "currently on the stack" mark. A stored 0 means "not computed yet," which is safe because every length is at least 1.

The path fillings are O(mn). Maximal square is O(mn). Maximal rectangle is O(mn) because each histogram is linear and there are m of them. The increasing path is O(mn), each cell expanded once.

TRAPS

- Filling unique paths and then overwriting the first row. Those cells are 1, and they have no top neighbor inside the grid.
- Treating an obstacle's edge as still reachable because you initialized the whole edge to 1 and never zeroed the suffix.
- Comparing maximal square against the number 1 when the matrix holds the character "1". They are not equal.
- Returning the side instead of the area.
- Memoizing a path that is allowed to stay flat. Without a strict increase the search cycles. The problem is strict.

Practice

Stay before chapter 25. Then before you leave the chapter. The histogram rectangle and the rising path are a second pass.

62. Unique Paths MEDIUM STAY
First row and first column are 1. Each other cell adds top and left.

64. Minimum Path Sum MEDIUM STAY
Edges are prefixes. Each other cell adds the min of top and left.

221. Maximal Square MEDIUM STAY
Character "1". Side is 1 + min of three neighbors. Return side * side.

63. Unique Paths II MEDIUM THEN
An obstacle writes 0 and cuts the edge. Start obstacle returns 0.

931. Minimum Falling Path Sum MEDIUM THEN
Min of the three cells above, clipped at the ends. Answer is the min of the last row.

85. Maximal Rectangle HARD SECOND PASS
Each row is a histogram of consecutive 1s. Reuse chapter 15.

329. Longest Increasing Path in a Matrix HARD SECOND PASS
Memoized search, four directions, strict increase, no cycle mark beyond the memo.

REWRITE FROM MEMORY

Rewrite unique paths and maximal square. Say "side times side" as you return.

Carry forward. Chapter 25 puts the path on a calendar and then hides the chapter numbers.

Four-week order and a mixed drill

Week by week, then a blank rewrite of fifteen templates, then a mock that hides the chapter.

You have the templates. The remaining work is the order you practice them, and a mock that refuses to tell you the chapter.

Four weeks

WEEK	CHAPTERS	WHAT YOU CAN DO BY SUNDAY
1	0–6	Index, classify, one pass, pairs, a map, both kinds of two indices.
2	7–11	A window, a prefix, a difference array, two passes, Kadane.
3	12–17	Sort, placement, binary search, a stack, a greedy scan, and you can name which one.
4	18–24	A matrix, a spiral, a mark, a rectangle sum, a staircase, a flood, a filling order.

Each weekday, read one chapter and rewrite its template the same day. Solve its Stay problems before the next chapter. Park Then problems on the weekend of that week. Second pass starts the week after you finish chapter 24, not before. The last two evenings of week 4 are the card below and the mock. If a Stay problem is still open, it outranks a new chapter.

Fifteen templates, from a blank file

1. Stock I. Profit uses the old minimum. Then the minimum may move.
2. Two sum. Map lookup of the complement, then insert.
3. Two sum on a sorted array. Too small moves l.
4. Container. Move the shorter wall only.
5. Read r, write w. Duplicates II compares with w − 2.
6. Variable window. Shrink with while. Length is r − l + 1.
7. Prefix frequency. Seed 0 → 1. Query sum − k. Then increment. Negative modulo repaired.
8. Difference array. +v at L, −v at R + 1, then one scan.
9. Kadane from nums[0]. Product uses temps. Circular rejects the empty wrap.
10. 3Sum. Skip duplicate i, and skip both ends after a hit.
11. Dutch flag. A swap with hi does not advance mid.
12. lowerBound. while lo < hi. ok sets hi = mid. Otherwise lo = mid + 1.
13. Next greater, stack of indices. Histogram with a 0 on both ends.
14. 2D prefix build, and the rectangle with the added-back corner.
15. Islands, mark on entry. Rotting oranges, mark on enqueue. A layer is one minute.

The night before

SIGNAL	PATTERN	USUAL COST	CH.
Summary of one side	One pass	n	**02**
Have I seen the complement	Map, insert after lookup	n	**04**
Sorted, discard one end	Two indices, ends	n	**05**
Overwrite in place	Read and write	n	**06**
Contiguous, shrink repairs it	Window	n	**07**
Sum K, negatives allowed	Prefix map	n	**08**
Range add, then read	Difference array	n + updates	**09**

SIGNAL	PATTERN	USUAL COST	CH.
Both sides, then combine	Left pass and right pass	n	**10**
Best non-empty subarray sum	Kadane	n	**11**
Pairs or intervals in order	Sort, then scan	n log n, or n^2	**12**
Value wants an index	Cyclic place, or XOR	n	**13**
Sorted half, or monotone answer	Half-open search	log n, or n log A	**14**
Nearest greater, window max	Stack or deque	n	**15**
Farthest reach, tank, one edit	Greedy scan	n	**16**
Rectangle sum	2D prefix	mn, query 1	**21**
Row-and-column sorted	Staircase from top-right	m + n	**22**
Component, minutes, word	Mark on entry, BFS layer	mn	**23**
Cell reads finished neighbors	Fill in order	mn	**24**

Mixed mock

Twenty-four problems, in an order that hides the chapter. Set a timer if you want the interview constraint: 45 minutes for one, or a sitting of four. Write the pattern name before you write code. The names are on the next page.

1. 1. Two Sum
2. 283. Move Zeroes
3. 200. Number of Islands
4. 209. Minimum Size Subarray Sum
5. 33. Search in Rotated Sorted Array
6. 53. Maximum Subarray
7. 54. Spiral Matrix
8. 11. Container With Most Water
9. 560. Subarray Sum Equals K
10. 739. Daily Temperatures
11. 75. Sort Colors
12. 56. Merge Intervals
13. 875. Koko Eating Bananas
14. 238. Product of Array Except Self
15. 1004. Max Consecutive Ones III
16. 73. Set Matrix Zeroes
17. 15. 3Sum
18. 994. Rotting Oranges
19. 167. Two Sum II - Input Array Is Sorted
20. 55. Jump Game
21. 304. Range Sum Query 2D - Immutable
22. 42. Trapping Rain Water
23. 128. Longest Consecutive Sequence
24. 74. Search a 2D Matrix

Mock answer key

Uncover this after you have named a pattern for each problem.

1. Map, lookup then insert. Chapter 4.
2. Read and write, then fill the zero tail. Chapter 6.
3. Depth-first search, mark on entry. Chapter 23.
4. Variable window, positive sums, shrink with while. Chapter 7.
5. Half-open search. One half of the rotation is sorted. Chapter 14.
6. Kadane, start at nums[0]. Chapter 11.
7. Four edges. Guard the last two legs. Chapter 19.
8. Pointers from the ends. Move the shorter wall. Chapter 5.
9. Prefix frequency. Seed 0, query before update. Chapter 8.
10. Monotonic stack of indices. Answer is the distance. Chapter 15.
11. Dutch flag. Do not advance mid after the swap with hi. Chapter 13.
12. Sort by start. Merge when start <= last end. Chapter 12.
13. Binary search the speed. Chapter 14.
14. Left products, then right products. Chapter 10.
15. Window of at most k zeros. The answer is the window length. Chapter 7.
16. First row and column as marks. Write back from the bottom-right. Chapter 20.
17. Sort, then two ends, skip duplicates. Chapter 12.
18. Multi-source breadth-first search. A layer is a minute. Chapter 23.
19. Pointers from the ends on a sorted array. Chapter 5.
20. Farthest reach. If i is past it, return false. Chapter 16.
21. 2D prefix. Add back the top-left corner. Chapter 21.
22. leftMax and rightMax, or the two-index collapse. Chapter 10.
23. Set. Start a run only when x − 1 is absent. Chapter 4.
24. Flat binary search. mid / n and mid % n. Chapter 22.

REWRITE FROM MEMORY

Rewrite the fifteen templates on blank paper. The mock is finished when each pattern name matches the key without a reread of the chapter.

Carry forward. There is no next template. When a new problem arrives, it enters through the recognition questions in chapter 17.